

VECTOR

APL 2000 Conference Report

Plus ...

- | | |
|------------------------------------|-----|
| • New Facilities in J 3.03 (Hui) | 52 |
| • Bohrer on Namespaces | 77 |
| • Cannon on Cyphers | 85 |
| • Lescasse on APL+Win and VBXs | 92 |
| • Burke on Operators in APL2 and J | 118 |



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.13 No.3 January 1997

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1996-97

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 01439-788385 CompuServe: 100331,644

Contents

		Page
Editorial	Duncan Pearson	3
APL NEWS		
Quick Reference Diary		5
Correspondence		7
Vector on the Internet	Ray Cannon	15
News from Sustaining Members	Gill Smith	18
The Education Vector	Ian Clark	21
APL Product Guide	Gill Smith	37
PRODUCT REVIEWS		
New Trace Facilities in J 3.03	Roger Hui	52
RECENT MEETINGS		
APL2000 at Orlando	Adrian Smith	62
Danish APL Group at Copenhagen	Gitte & Stefano	74
GENERAL ARTICLES		
Import/Export and the Wealth of Namespaces	Douglas Bohrer	77
How to Ride a "BiCypher"	Ray Cannon	85
TECHNICAL SECTION		
Hacker's Corner: Clipboarding APL Arrays	Adrian Smith	90
APL+Win and VBXs	Eric Lescasse	92
J and APL Operators	Norman Thomson	112
APL and J (3): Operators		
Reduce, Scan, Outer Product	Chris Burke	118
At Play with J: Extended Integers	Gene McDonnell	127
Using a Defined Adverb to Trace Execution in Tacit Programs in J	David Alis	136
Index to Advertisers		143

Monthly APL+Win Training from Uniware On Line !

Available WorldWide

If you are an APL user running Windows,

If you want to exploit it to the utmost,

If you want to learn in depth Windows programming,

If you want to get a large library of **reusable** APL+Win software,

If you want to be able to use the 32-bit Windows API,

If you want to see a complete Windows application developed,

If you want to learn about DDE, VBX, OCX, ActiveX, OLE, ODBC, ...,

If you want to interface APL+Win and Delphi, Excel, ...,

If you want to learn how to print easily from APL+Win,

If you want to learn about Relational Database Programming in APL,

If you want to have fun with Virtual Reality and with the Web in APL+Win,

Do not hesitate one more minute!

Subscribe to our

Monthly APL+Win Training Program

Try it free from our site:

<http://www.uniware.fr/uk>

and print all the details and Subscription Form about this program!

If you subscribe, you will be able to download every month:

- a 50 pages (or more) WinWord APL+Win training document
 - an APL+Win workspace full of **reusable** code
 - often, additional utilities and tools

Subscription available directly from Mr. Eric Lescasse at Uniware

Fax you Subscription Form to Mr. Eric Lescasse - Uniware - fax: 33.1.40.90.04.11

(or for USA users to our local US distributor: Mr. Eric Baelen - APL2000 - fax: (609) 734 9644)

APL+Win is a TradeMark of APL2000, Inc. "Monthly APL+Win Training" is Copyrighted by Mr. Eric Lescasse

Uniware - Tour Neptune - 92086 PARIS LA DEFENSE CEDEX - France - tel: 33.1.47.78.78.00 -
fax: 33.1.40.09.04.11 - Compuserve: 70731,3233 - email: lescasse@uniware.fr

Editorial

by Duncan Pearson

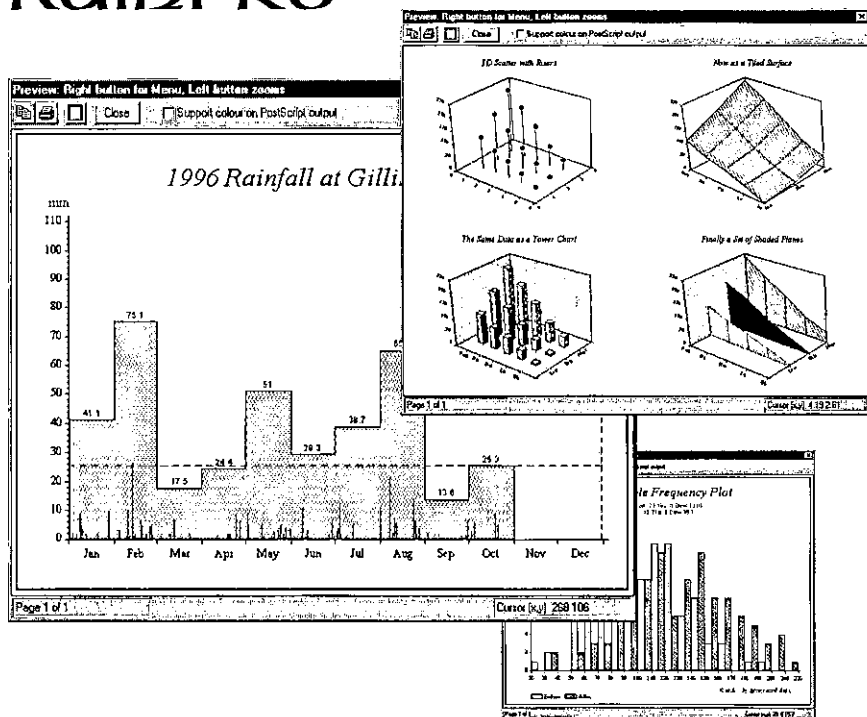
As promised in the last issue of Vector we have from the irrepressible Adrian Smith a report of the APL2000 user conference in Orlando together with news of the recent meeting of the Danish APL group kindly reported by Gitte Christensen and Stefano Lanzavecchia. Furthermore, by the time you read this we will have had the first BAA Internet meeting, a meeting not just *about* but also *on* the Internet. Again a full report will be included in the next issue, and if it is successful no doubt more will follow along the same lines.

On the technical front we have from *Eric Lescasse* a good introduction to the use of VBX and OCX controls from APL+Win. The particular example is a grid control and perhaps will go some way to easing the frustration felt by many APL+Win users who acutely feel the lack of a grid control built into the interpreter. Also I am pleased that at last someone has answered the plea for discussion among APL developers of the ways in which namespaces might be implemented, enhanced or used. We have an article from *Douglas Bohrer* which explores some of the possibilities offered by DyalogAPL and offers some ways in which he believes that it might be improved. I hope that this promotes some discussion. I know that there are some who having moved to using namespaces feel that they could not move back. Others, after initial interest, find that they do not offer what they had hoped and so go back to tried and tested methods of workspace organisation such as function files. We would like to hear from you.

In J we have the third of *Chris Burke's* excellent articles on J for the APL programmer. I have found them most instructive. It is good at last to see the differences highlighted and perhaps surprising to see where the similarities lie. This was clear to attendees of Chris's tutorial at Lancaster, some of whom were surprised at the length of time that Chris spent in APL. It is also the subject of an article by *Norman Thomson* on the way the use of operators in J has influenced the way he writes APL.

We also have a remarkable coincidence, with an article from *David Alis* on a way of tracing the execution of long strings of J arriving a few days before a submission from *Roger Hui* describing a new foreign conjunction that will provide exactly those facilities. We have still included the article from David Alis as the approach is of interest and because it demonstrates some subtle points about the rank of functions. Roger assures me that the new version will be available from mid-January.

RainPRO



Publication-quality charting for
the APL developer

- high quality graphs with minimal programming
- totally customisable for those complex one-offs

Available for Dyalog APL/W and APL+Win

Price £250 (\$400) includes on-line help, printed
User's Manual, and one year's telephone support.

Causeway Graphical Systems Ltd.
The Maltings, Castlegate
MALTON, North Yorkshire
YO17 0DP

Tel: +44 (0) 1653 696760
Fax: +44 (0) 1653 697719
Video: +44 (0) 1653 693006
causeway@compuserve.com

Quick Reference Diary 1997

Date	Venue	Event
June 23-24	University of Toronto	J User Conference
October 22-24	Orlando, Florida	APL2000 User Conference

Dates for Future Issues of VECTOR

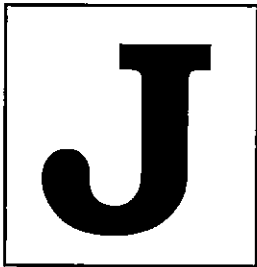
	Vol.13 No.4	Vol.14 No.1	Vol.14 No.2
Copy date	8th March	6th June	5th September
Ad booking	15th March	13th June	12th September
Ad Copy	22nd March	20th June	19th September
Distribution	April 97	July 97	October 97

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.



*In case you thought we were
having too much fun...*

...we have been pretty busy with J 3.03!

The new Socket Driver in J Release 3.03 provides support for using sockets under Win 95/NT and UNIX, so you can now use J to access the Internet, or connect to any machine running TCP/IP.

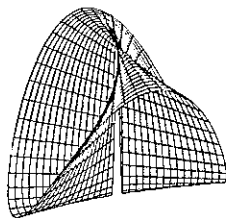
Printing facilities have been completely revamped - allowing you to mix text and graphics with fine control over spacing and justification. This ad was produced entirely from a J program!

We improved the user interface in a number of ways - context sensitive help jumps to the current Dictionary entry; a customizable observe facility lists the steps taken by the interpreter as it runs your code. We simplified explicit definitions, and extended J error messages.

We've added a tutorial facility, expanded the on-line helps, and included the J Primer - the beginner's book for the rest of us.

We changed the packaging, so that all editions now include the full J system, and made all Windows and Mac systems available on-line from our Web page.

*Evaluation copies of J for
Windows and for the Mac are
available from our Web page.
Also look for J Runtime
systems, texts of the system
manuals, including the new J
Primer and J Phrases, papers,
tutorials and demo scripts.*



Distributed by:
Strand Software Inc.
19235 Covington Court
Shorewood, Minnesota
USA 55331
Tel (612) 470-7345
Fax (612) 470-9202

www.jsoftware.com

CORRESPONDENCE

«écoeuré»

From: Gérard Langlet

12 November 1996

Among the at-least 35 pages devoted to APL96 at Lancaster in the last issue of *Vector* (Vol.13, No.2, Oct. 1996), I find no mention of Dr. Michael Zaus' conference about *Parity Logic*, although Mike was invited jointly by BAPL and APL Club, Germany. I do not even speak of my own conferences about the *Least-Action Principle* and *Chaos Theory Revisited*. Are we included within the «etc.» p.77 of R. Procter's paper mentioning — almost with disgust — the academic side of the APL/J community? In contradiction with what is written in the same page 77, presentations of submitted papers were not “generally poorly attended” (at least Mike's and mine). I also remember that at APL94 in Antwerp, I had to double my workshop about Binary Logic (was APL96 as quoted p.82 — the “best conference in terms of excellent presentations and informative workshop and tutorial content...”? As an *affidatus* of APL conferences since Pisa in 1975 except Sydney and San Antonio, I really doubt it).

Was this attitude deliberate and partial, or simply the consequence of a lack of culture or a lack of interest — which corresponds to the same thing? The only word that fits what I feel is «écoeuré» in French (the English equivalents that are given e.g. in the Harrap's for the translation are indeed inadequate).

Will APL at last become a research tool in Computer Science (a genuine wish, expressed by Ken Iverson a long time ago) and even in physics, or appear as a Duck tool (I have nothing against Dyalog especially, working in fact with many implementations, including J).

Dr. Gérard A. Langlet
SCM/LIT, CEA-Saclay
F-91191-Gif sur Yvette,
France

[As *Vector* goes to press, we hear the sad news that Gérard died on 22nd December. Dick Holt posted the following to `comp.lang.apl` on Christmas Eve: “The APL community deeply mourns the death of Gérard Langlet. His witty, learned, and creative work will long be treasured by his colleagues. Speaking personally, I'll remember, and miss, him for the rest of my life.”]

Antecedent Precipitation Index (again)

From: Roger Stokes

10 December 1996

In his recent letter (*Vector* Vol.13 No.2, "While in J ...") Nick Cox asks whether a quicker concise version can be produced of his J verb "api2", which is reproduced here:

```
api2 =. 3 : 0
NB. antecedent precipitation index NJC 15 April 1996
NB. k api p
:
+/"1 (* x. &^ @ i. @ - @ #) \ y.
)
```

```
0.5 api2 1 2 3 4
1 2.5 4.25 6.125
```

It can be seen that, given a vector p and a scalar k, then (k api2 p) computes the vector a from the relationship (which I restate as):

$$a[0] = p[0]$$

$$a[i+1] = p[i+1] + k \times a[i]$$

Nick Cox notes that api2 is slower and needs more space than his longer version:

```
api1 =. 3 : 0
NB. antecedent precipitation index NJC 15 April 1996
NB. k api p
:
k =. x.
p =. y.
api =. 0
i =. 0
while. i < #y. do.
api =. api , (i { p) + k * {: api
i =. i + 1
end.
}. api
)
```

The longer version api1 implements the recurrence relation directly, while the shorter api2 implements the derived relation:

```

a0 = p0
a1 = p1 + k*p0
a2 = p2 + k*p1 + (k^2)*p0          etc

```

that is, each item of a is a polynomial in k. The two versions give the same results:

```

0.5 api1 1 2 3 4
1 2.5 4.25 6.125

```

Performance of the two versions was compared, at 3 different lengths for the vector argument p (with p=. i. 10, p=. i. 100 and p=. i. 1000). Times in seconds on my machine, at the 3 argument-lengths, are:

	10	100	1000
api1	0.05	0.43	4.35
api2	0.02	0.53	-

api2 gives "out of memory" at argument length 1000, and is slower at argument length 100. The first improvement which might be made is to do the summing as soon as possible, rather than leaving it to the end. This means that we can avoid building an n by n matrix as an intermediate result. The verb I differs from api2 only by absorbing the +/ into the verb which is given to the \ operator:

```

I =. 4 : ' (+/ @ (* x. & ^ @ i. @ - @ #)) \ y. '
0.5 I 1 2 3 4
1 2.5 4.25 6.125

```

The timings for the three versions are:

	10	100	1000
api1	0.05	0.43	4.35
api2	0.02	0.53	-
I	0.02	0.52	59.03

Evidently I is no faster than api2, but it does run to completion. The next improvement can be made by using J's primitive p. to evaluate all the polynomials, rather than the programmed equivalent of api2. The algorithm is essentially the same, although reversals of ordering are needed to conform with p. .

```
II =. 4 : ' |. (p. & x.) \. |. y. '
```

```
0.5 II 1 2 3 4
1 2.5 4.25 6.125
```

Verb II is faster than I, although still not as good as api1 at the longer argument lengths:

	10	100	1000
api1	0.05	0.43	4.35
api2	0.02	0.53	—
I	0.02	0.52	59.03
II	0.01	0.14	9.30

Even so, algorithms api2, I and II are inherently inefficient because each term in the result is computed independently: all that work is thrown away when with just a little more work it could be used to give the next term as well! In relation to the argument-length, the execution time of api2, I and II looks like a square-law, whereas that of api1 looks linear.

Let us return to the serial algorithm of api1. If we use a J operator to replace the programmed control of api1, the iterator $\wedge$: (power conjunction) or the insertion operator / both look promising. Both were tried, but / gave the better result.

With / we will need a dyadic verb, u say, to insert between each item of argument p. Given a value for k:

```
k =. 0.5
```

then a first approximation to u would be:

```
u =. 4 : 'y. , (x. + k * {: y.)'
```

which closely resembles the key line of api1. Inserting u between each item of (the reverse of) argument p gives the required result:

```
u / |. 1 2 3 4
1 2.5 4.25 6.125
```

Now we need to supply k as an argument rather than as a global variable. k can't be an argument to the dyad u because both of u's argument-places are spoken for. If we first define verb v as a tacit equivalent to u:

```

      v =. ] , (+ (k & *) @ {: @ ])
    v / |. 1 2 3 4
1 2.5 4.25 6.125

```

then we can parameterise v to make an adverb, A, say, and the argument of A will be the value k.

```

      A =. 1 : ' ] , (+ (x. & *) @ {: @ )]'
    (0.5 A) / |. 1 2 3 4
1 2.5 4.25 6.125

```

Combining A with the insertion operator, and the needed reversal of the argument p we get an adverb, III say, which is structured as a train of 3 adverbs:

```

      III =. (1 : ' ] , (+ (x. & *) @ {: @ )]' ) / (@ |.)
    0.5 III 1 2 3 4
1 2.5 4.25 6.125

```

The timings of all versions are:

	10	100	1000
api1	0.05	0.43	4.35
api2	0.02	0.53	-
I	0.02	0.52	59.03
II	0.01	0.14	9.30
III	0.02	0.09	1.37

Algorithm III is faster than api1 at all argument lengths tried.

Roger Stokes

rstokes@dial.pipex.com or qk57@dial.pipex.com

While still in J ...

From: Norman Thomson

24 November 1996

Nick Cox in a letter (*Vector* Vol.13, No.2, p.10) compares a Fortran-like J program to the functionally equivalent one-liner `+/ "1(* x.&^@i.@-@#)\y.` and asks whether a quicker and more concise J script can be produced. As far as the second requirement is concerned, Nick should observe that he is in fact evaluating a polynomial in his variable *k* (*x.* in the above), and a functionally equivalent verb to the above is `>@(#.&.><\).` This is at least as fast in execution as either of Nick's programs. Space depends of course on the length of the series *y.* — with 1000 terms, the polynomial version requires one-eighth of that needed by Nick's one-liner (2Mb vs.16MB), although naturally the step by step version is superior to either (30KB).

As a tail-piece, timing experiments such as the above can be interrupted by Ctl-Shift-C. This piece of information, which I have not been able to find in any of the J manuals or Help material, was passed to me by Bill McLean.

British J Association?

From: Don Mattern

18 December 1996

When are you going to change your name to British J Association? We subscribed to *Vector* to receive APL information. How can you assume APL'ers are going to learn a new language? APL symbols may be Greek, but they are largely intuitive, whereas what is intuitive about `@#%"&:?` J is a concession to ASCII and as someone said is so much 'comic book invective'.

It would be interesting to find out if corporations are switching from APL and other languages to J. It seems to us that J is the province of academics and theoreticals. R. G. Brown asks why isn't APL used more widely. If APL scares people because of its symbols, what do you think J is doing?

We note that IBM has not endorsed J, nor the learned physicist, Gérard Langlet. If the bulk of *Vector* is going to be on J, do not renew our subscription.

Don Mattern
mattern@wco.com

Leaving the Nest

From: R.G.Brown (MMSI)

20 December 1996

Recently I have been working with a couple of very *avant garde* organizations that seem to use nested arrays because they are possible. I should like to enter an alternative view, based on experience with using APL since the day after Al Rose and Larry Breed had the first prototype up and running on an IBM mainframe.

My specific beef has to do with passing arguments to, and results from, functions. It has long been my practice that any function a user sees is nihiladic — no arguments and no explicit results. Many subroutines, invisible to the user, do require arguments: lists of products, column headings, row headings, index to the Help file, various kinds of data (which may range from scalars to arrays of rank 3 or 4).

These other organizations, staffed by people much younger than me and therefore more amenable to modern procedures, seem to prefer to build elaborate nested arrays with all the arguments, and to generate results as nested arrays which don't have the shape you think they have when you look at them.

I see no point in specifying the index to the Help file — it is always $1+t\ t$ where $t\ t$ is a global variable with lots of $\square$ AI kinds of information, including a pointer to where this particular function (main program or subroutine) is stored in the file of programs. Row headings and column headings are easy to construct (or were before they got rid of $\square$ EDIT) as a character array of some suitable shape which is stored in a file of program constants.

My workspace has only 16MB, so I differ from the Supreme Satan at Microsoft in my willingness to clutter up the workspace with functions and variables that won't be used for a couple of hours. Call what you need from file when you need it, and get rid of it afterwards. Conserve the workspace for data that will do the user some good in running his business. This attitude may have been inculcated when my workspace was a punched card with 80 columns, later going up to a 1401 which had a whopping 3K of space.

When a subroutine is closely coupled with its main program I see no harm in using a global variable for data. I think it may still be true that if I don't modify the data no second copy is made to fill up the workspace. If data are passed as an argument there is a greater danger of *WSFULL*.

It seems to me that a subroutine using nested arrays requires considerable work before it is called to assemble each of the eggs in the nest with precisely the right

punctuation, like an old-fashioned FORTRAN program. Then the first thing the subroutine does is to disassemble the nesting to get back to the elements which were available to build the nest in the first place. Somehow that does not appeal to my sense of fitness.

I find that complicated nested arrays are hard to hold in my head, hard to write routines to build them, probably time-consuming in execution. Therefore I suggest that the profession might give some careful consideration to passing a vector of numbers which the program can interpret to go get what it wants out of files. It certainly makes maintenance of the contents of those arrays much easier. With some of my handy-dandy development tools editing stuff in files becomes self-documenting which helps other people to follow what has gone on.

Vector on the Internet

progress report by Ray Cannon (100430.740@compuserve.com)

In the last issue of Vector, I stated what I was hoping to do with Vector's web site (<http://www.vector.org.uk>) in particular the product guide. I'll now tell you what has been achieved.

(As before, I will call the printed edition *Vector* and the web version vector.org.)

Product Guide

At the top level is the product guide menu:

<http://www.vector.org.uk/product.html>

This allows you to select the section of the product guide you are interested in via a sub-index. In addition there is a full listing of all companies, and a BOOKMARK page.

The bookmark page contains *all* the (fully qualified) links to each individual product guide entry plus any additional link or e-mail address. I keep a copy of the BOOKMARK page locally on my hard disk. This gives me very fast access to every item in the guide. I used the e-mail entries in the bookmark page to create an (albeit incomplete) product guide mailing list.

Each sub-index page consists of a "quick index" and an alphabetical list of company/organisation with links to the product guide entry.

Within a product guide entry, the web page is normally divided into three or more areas. The first area is the "contact" section. Here are listed (when known/applicable) the company's name, address, contact name, phone number, fax number, web address, and FTP site.

Subsequent areas contain individual product entries much as in Vector. The last area contains the usual return links, disclaimer, minder robot notification link (see below), copyright notice, etc. There is also a button marked "update".

This button will, when activated, redisplay the web page but with an "update" button in each section. Pressing a section's update button will create a new web page containing a dynamically built form with this section's details pre-filled. This form will allow the owner to modify the section entry and then submit it to

me to so that I can update the original web page (and pass it on to Gill to update the printed Vector entry). Please feel free to try this out (just put "Testing" in the authorisation field if you don't want the section updated).

The complete "UPDATE" facility is driven by some programs I have written in PERL, using the CGI facilities. Please let me know of any problems with them.

Minder Robot

Many of the vector.org pages now have a section which allows the viewer to set up a "minder robot" which will notify the user whenever the web page is modified. To activate this service, you need to give the "robot" your e-mail address. "Minder Robot" is a free service not connected to Vector or the BAA in any way. This service can be activated to report on ANY page on the web, not just those with links to it. Currently the robot will visit each site it is minding about once a week to check for changes. I find it very useful.

Things to come

At present, neither the complete index to Vector nor the Product Guide within vector.org has a search engine. I am planning to implement one in Perl when I get time. (I am also looking at JavaScript, but currently this is not supported widely enough to be useful; for example it works with Netscape Navigator but not (correctly) with Microsoft Internet Explorer.)

The "last modified date" facility is working on all pages in the root directory of the Vector web site. However, it currently does not work on the product guide entries (which are located in a sub-directory). I hope to be able to find (and fix) the cause of this error.

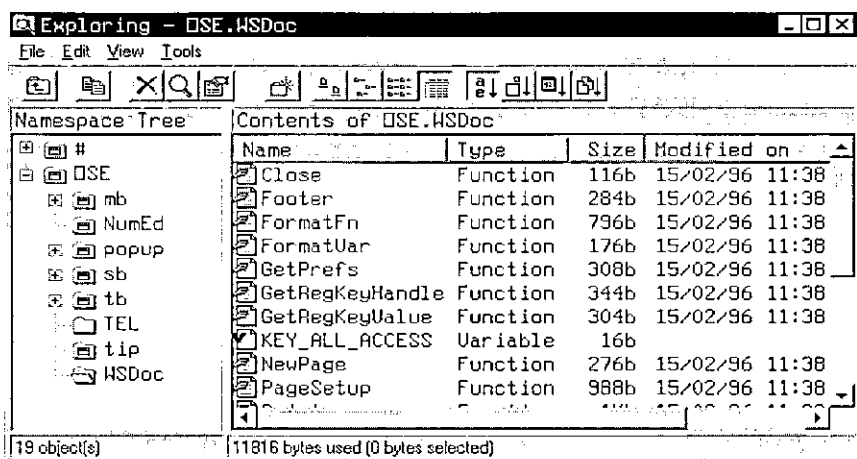
And Finally...

If you have any suggestions to what you would like to see (or not see) in vector.org (or in Vector for that matter), please let us know. One request was for the whole of the product guide to be a single page so it could be easily searched, or downloaded. Do you want to be able to put your CV up in vector.org? Do you want more graphics? I have NOT used any in the past to keep transfer times down to a minimum, but it makes the pages look very dull compared to Vector's bright covers!

And finally, can I thank all those of you who have already sent in suggestions and feedback.

dyalog APL

The Definitive APL for Windows™



Dyalog APL Namespaces let you ...

- **Organise** your workspace into self-contained sub-systems
- **Encapsulate** functions and variables *within* GUI objects
- **Isolate** utilities from your application code

plus The new Version 8 **Explorer** lets you browse namespaces, drag-drop objects from one to another, and a whole lot more.

That's why Dyalog APL/W remains the **professional** choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



News from Sustaining Members

Compiled by Gill Smith

Dyadic Systems Limited

The Beta Test release of Dyalog APL/W Version 8.1 was shipped on December 23rd, with the production release scheduled for February.

Version 8.1 provides extensive support for OLE Automation, allowing APL to act both as a client and as a server. See Vector Vol.13 No.2 for details.

Dyalog APL with OLE technology offers a new range of opportunities for APL developers. Not only can you deliver complete packaged GUI applications in APL, but you can now market stand-alone software components that can be incorporated into other products which are constructed with other languages.

Version 8.1 also includes support for TCP/IP. This is implemented by a new *TCPSocket* object that is controlled using the standard Dyalog APL object management functions (*⌈WC*, *⌈WS* and so forth). Like any other Dyalog object, a *TCPSocket* is of course a namespace in its own right.

The *TCPSocket* object allows you to communicate with other TCP/IP applications running on any computer in your network, including the World Wide Web. It supports both Stream and UDP connections and, like the underlying WinSock API, is predominantly event driven. The *TCPSocket* object supports character, raw, and internal APL data formats and also provides the mechanism for client/server operation between two Dyalog APL workspaces.

Finally, Version 8.1 includes *dynamic functions*. A Dynamic Function (operator) is an alternative function definition style suitable for defining small to medium sized functions. Dynamic functions support automatic localisation, nested functions, and a simple but powerful control structure called a *Guard*. See Vector Vol.13 No.2 for details.

In addition to these major new features, Version 8.1 includes other useful enhancements; and there is a parallel release 7.3 for Windows 3.1. For further information, consult the Dyalog APL web site at www.dyadic.com or contact your Dyalog APL distributor.

Insight Systems / Adaytum Software

Insight Systems and Adaytum Software are now properly "wired"; our internal Microsoft Mail system is linked to the Internet. Or so we think — the technical glitches had not all been ironed out as this is written, but we are assured that all will be OK when this appears in print. To be sure, please check our Web sites before sending E-Mail. Addresses to note are:

Web sites:

www.adaytum.dk	Adaytum Denmark and Insight Systems
www.insight.dk	(these two addresses are aliases for the same domain)
www.adaytum.com	Our US-Midwest office in Minneapolis/St.Paul

E-Mail addresses:

info@insight.dk	For queries regarding Insight's products and services
support@insight.dk	To report problems or request assistance if you are a user of our products

Once the pilot projects in Denmark and the USA are completed, our Bristol office will become **adaytum.uk** and our new Paris office will be **adaytum.fr**.

People

One of the people you will be able to reach (as jb@insight.dk) is Jacob Brickman, who joined the Adaytum Planning team on November 1st as a senior developer.

Products: Adaytum Planning

The 32-bit version of Adaytum Planning, based on Dyalog APL version 8.0, for Windows 95 and Windows NT, is scheduled for release at the end of January 1997. A new version of our Windows Reporter product, based upon Causeway's Rain and NewLeaf printing technologies, will be available in both 16- and 32-bit versions in April.

Products: SQAPL

There are no new product announcements this quarter; we are busy programming but not ready to announce release dates for our OLE frontend for APL (for Visual Basic, Delphi etc.) or the new SQAPL ODBC Server for Windows NT.

Causeway Graphical Systems Ltd

This has been a quarter of steady progress on two fronts. It has been good to clear the backlog of requests for *RainPro* and *NewLeaf*, both of which have benefitted from beta feedback, and which will remain stable in their November versions for the next few months for both Dyalog and APL+Win. The few minor bugs remaining are being noted on the Causeway Web site as users flush them out, so that you can avoid them or fix them in place as you prefer. The only major last-minute change was to add `Δspoolpath` as a global in the *leaf* namespace, as otherwise NewLeaf simply used the current working directory as its work area; this can be a dangerous habit on a networked installation where the user may not have write access to the working directory!

Work in progress for the next NewLeaf version includes tables which fold across as well as down, and specific optimisation for the LaserJet II printer. The next release of RainPro will support a wide range of 'chart events' and methods so that graphs may be modified in-place by the users, and pop-up dialogues can be programmed easily to allow modification of headings, labels, colours, fonts and so on.

On the application-development side, Duncan Pearson and Jon Sandles have been working steadily towards the next SAP release date. The mixed C++ and J environment continues to perform well, and is becoming much easier to work with as all the complexity of the linkage is now well-buried in our own increasing collection of OCXs. This brings the technology back within the grasp of the average APLer (which the black art of OCX development certainly is not), as it is now accessible from Visual Basic as well as from C++. As soon as we have time to pause and take stock, we will review and complete our OCX library and consider how to make it available to the J community.

Having survived in one (quite small) office unit for nearly two years, we finally spread ourselves out just before Christmas into much more comfortable surroundings. The address, phone and fax numbers and emails remain as they were, except that Compuserve have finally implemented personal addresses, so if you feel more comfortable sending mail to causeway@compuserve.com, this now works. Of course the old number remains fine too.

THE EDUCATION VECTOR

January 1997

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.13 No.3. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Sylvia Camacho, 11 Auburn Road, Redland, BRISTOL, BS6 6LS. Tel: 0117-973 0036.

Contents

Functional Programming in J (III)	Howard Peelle	22
J-ottings 12	Norman Thomson	29

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 01388-527190
Email: 100021.3073@compuserve.com

Introduction to Functional Programming in J (Part III)

by Howard Peelle (hpeelle@educ.umass.edu)

This is Part III of a four-part article. Part I (in Vector 13.1) presented fundamentals of J in a framework of functional programming strategies, with examples expressly aimed toward developing an illustrative program to generate prime numbers. Part II demonstrated how to develop the program directly, while characterizing a J programmer's thinking. Part III continues development of the same program with higher-level strategies. Part IV will consider alternative algorithms for iterative, recursive, and array-processing programs and will end with meta-strategies for packaging and documentation.

8. Simplify, Optimize, Generalize Programs

An often overlooked strategy involves looking back at a program after its development and rethinking it. This is an opportunity to simplify or optimize or generalize the program (and subprograms). To 'simplify' here means to make a program shorter and/or more readable. To 'optimize' means to increase speed and/or reduce space required for computations; that is, improve efficiency. To 'generalize' means to design the program to apply to many (preferably all) cases and/or higher dimensional arrays.

Simplify

First, consider some ways to simplify the programs developed previously. For instance, the Primes program (from section 6):

```
Primes
Pos If Prime"0@Pos
```

NB. Display program

One way to simplify this definition is to enter a list of integers as input, thereby removing the need to generate them (twice, using Pos) within the program. Thus:

```
Primes =. ] If Prime"0
```

NB. Primes is

NB. Input If (Prime Rank 0)

Example of its use:

Primes 1 2 3 4 5 6 7 8 9 10 11 12
2 3 5 7 11

NB. Or Primes Pos 12

Another slight simplification involves using another form of composition in J — called a “hook” — where two verbs are written together in an isolated sequence. The (dyad) verb on the left uses the input and the result of the (monad) verb on the right. For example, (* >:) is a hook for the input Times (*) the Increment (>:) of the input. Give this hook a name:

H =. * >:

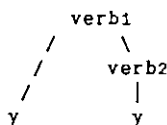
NB. H is Times Increment

Test it with one input:

H 7
56

NB. 7 * >: 7

In general, for one input y, a hook (verb1 verb2) y is equivalent to y verb1 (verb2 y), as shown in the diagram:

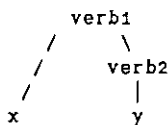


A hook may be given two inputs. For example:

10 H 7
80

NB. 10 * >: 7

In general, for two inputs x and y, a hook x (verb1 verb2) y is x verb1 (verb2 y). Diagram:



Now simplify the definition of Primes by removing one symbol (]) and forming a hook of the two verbs If and Prime"0 like this:

Primes =. If Prime"0

The example of its use is the same as before:

```
Primes 1 2 3 4 5 6 7 8 9 10 11 12
2 3 5 7 11
```

Another more substantial simplification can be achieved by noticing that the actual divisors aren't needed to detect a prime. Just count them. So, create a simpler subprogram to return 1s marking the divisors (as shown earlier in the first section):

```
Divisors =. Pos Divides ] NB. Divisors is Pos Divides Input
```

And revise subprogram Prime accordingly to Sum (+/) the 1s:

```
Prime =. +/@Divisors = 2: NB. Prime is
                        NB. (Sum Atop Divisors) Equal Two
```

Main program Primes gives the same result as before. A further simplification arises by noticing that the input integer is always a divisor of itself. Instead of Pos producing 1 to n, use Integers (i.) to produce 0 to n-1 for the list to divide the input. Don't worry about the 0 — for instance, 0 Divides 12 is 0 (false) because the remainder 0 | 12 is 12).

So, revise Divisors:

```
Divisors =. i. Divides ] NB. Divisors is
                        NB. Integers Divides Input
```

And revise Prime to use 1: (which returns the constant 1) instead of 2::

```
Prime =. +/@Divisors = 1: NB. Prime is
                        NB. (Sum Atop Divisors) Equal One
```

Similarly, notice that 1 is always a divisor of the input integer. Another simplification could remove 1 (and 0 as well) from the list generated by i. in Divisors and then replace 1: with 0: in Prime. But this definition (not shown here) admits 1 as a prime incorrectly, so it would have to disallow 1 in the input items. In either case, the main program should be used like this:

```
Primes 2 3 4 5 6 7 8 9 10 11 12 13
2 3 5 7 11 13
```

This suggests defining another utility verb to produce a list of any consecutive positive integers such as 2 to 13. Thus:

```
To =. }. i.8>: NB. To is input Drop
                NB. (Integers Atop Increment)
```

This definition is a hook (introduced above) of two verbs. The first verb, called Drop (.), removes items from a list; it uses two inputs — the left input is how many items to drop and the right input is the list of items. The second verb is a composition of Integers (i.) Atop (@) Increment (>:), producing integers from 0 to its input. So, x To y drops x items from the list of integers from 0 to y — in effect, listing the integers from x to y. For example:

```
2 To 13
2 3 4 5 6 7 8 9 10 11 12 13
```

From now on, use this for convenience like this:

```
Primes 2 To 13
2 3 5 7 11 13
```

Optimize

All these simplifications made the programs more efficient as well. J offers helpful tools for measuring such optimization efforts. For instance:

```
Time =. 6!:2          NB. Define Time
Space =. 7!:2         NB. Define Space
```

Now the time and space required to execute a program (or any expression) can be determined:

```
Time 'Primes 2 To 13'
0.0.027                      NB. Seconds (approximate)
Space 'Primes 2 To 13'
1720                         NB. Bytes
```

Benchmarks for a larger number of primes:

```
Time 'Primes 2 To 1000'
5.55
Space 'Primes 2 To 1000'
85756
```

Comparing these results with Time 'Primes 1000' and Space 'Primes 1000' for the earlier developmental definition of Primes (in section 6) reveals a factor of about 2 speedup as well as a reduction in space of about 9%. Note: These benchmarks were obtained on a 486 PC running JFW (J Freeware for Windows) Release 3.

Further, optimize the program by replacing subprograms with primitives (as shown in section 7):

```
Primes =. Primes f.
```

NB. Primes Fixed in primitives

Now compare benchmarks:

```
Time 'Primes 2 To 1000'
4.30
Space 'Primes 2 To 1000'
85756
```

This optimized program performs faster in the same space. (See Part IV for more optimization efforts for alternative algorithms.)

Generalize

J programs often generalize easily — sometimes without revision. Although the programming task here does not lend itself to significant generalization, note that program `Primes` will work (as is) for a list of integers (greater than 1) input in any order. For example:

```
Primes 6 3 8 2 7 11 10 4 12 5 13 9
3 2 7 11 5 13
```

NB. Unordered list

NB. Unordered primes

The result can be ordered — at additional expense — using the primitive `/:` (called `Sort`) embodied in a utility verb as follows:

```
Sort =. /: ]
```

NB. Sort is input Sort Input

```
Sort Primes 6 3 8 2 7 11 10 4 12 5 13 9
2 3 5 7 11 13
```

NB. Ordered primes

Further, consider using program `Primes` with a table or a 3-D array of integers input (although this is admittedly unusual). First, here are some ways to create such higher dimensional arrays. J's primitive verb `Shape ($)` can be used to create an array of any shape (given on the left) made up of any items (on the right). For example:

```
3 4 $ 2 To 13
2 3 4 5
6 7 8 9
10 11 12 13
```

NB. 3 by 4 table

And:

```

      2 3 4 $ 2 To 25
2    2 3 4 5
6    6 7 8 9
10   10 11 12 13

14   14 15 16 17
18   18 19 20 21
22   22 23 24 25

```

NB. 2 by 3 by 4 array

NB. (Two 3 by 4 tables)

NB. Blank line between

Note that (even though these examples do not illustrate it) items may be in any order, are repeated cyclically if there are too few, and are truncated if there are too many. Another way to create ordered arrays (such as above) uses the Integers primitive verb (i.) with a list input. For example:

```

      i. 3 4
0    0 1 2 3
4    4 5 6 7
8    8 9 10 11

```

NB. Integers in 3 by 4 table

Other verbs may be applied to this table, such as:

```

      2 + i. 3 4
2    2 3 4 5
6    6 7 8 9
10   10 11 12 13

```

NB. 2 Plus Integers 3 by 4

NB. (parallel addition)

A 3-D array would be expressed similarly: 2 + i. 2 3 4. Note that longer lists input to Integers (i.) may create higher dimensional arrays beyond the computer's limits (as encountered in beginning of section 6 where i. 1 2 3 4 5 6 7 8 9 10 11 12 created a 12-dimensional array with 479001600 items).

The primitive verb Ravel (, used with one input) makes any such array into a list:

```

      , 2 + i. 3 4
2 3 4 5 6 7 8 9 10 11 12 13

```

NB. Ravel 2 Plus Integers 3 by 4

So, to use an array with the Primes program, Ravel the input first:

```

      Primes , 2 + i. 3 4
2 3 5 7 11 13

```

NB. Primes in Ravel 3 by 4 table

So there is no need to change the program.

Next, in Part IV, we will consider alternative algorithms for generating prime numbers in recursive, iterative, and array-processing programs.

University of Massachusetts
Furcolo #10
Amherst, MA 01003
Tel: (413) 545-1114

Reference

- [1] Iverson, K. E. J *Introduction and Dictionary*, Iverson Software Inc., Toronto 1995
-

J-ottings 12

by Norman Thomson

And now for something totally different! Previous J-ottings have been about language matters; this one aims to provide an easy introductory tutorial for windows programming as provided in freeware J. Although this is fully documented in the *User Manual*, the first-time reader may well find that the complexity of detail there is somewhat daunting, thereby disguising the underlying broad ideas. Taken a step at a time, these are not as formidable as they look at first sight.

The primary assumptions are that the system-supplied `profile.js` has been run, and that the reader wishes to pass a J application to third party users through mechanisms like dialog boxes, list boxes and so on, with which computer users nowadays are all intuitively familiar. A supplied script called `wp.js` supplies seventeen examples of these, and running this script (choose Run Window from the Tool Bar, then run `''` from the execution window, pressing <Esc> after each window is displayed) gives a comprehensive demonstration of the facilities available, thereby obviating the need to fill many column inches here with bitmap illustrations. The script `wp.js` on my system is in `\examples\archive` and can itself be used as a tutorial. What follows here is an even more elementary description of the ground rules.

Windows programming in J is about passing seemingly arcane operating-system-like commands to a processor called the Windows Driver (`wd` for short). Although you still use J verbs, do not expect to find the sort of unities, subtleties, and expressive succinctness which make the J language, as with APL before it, such a source of programming delight.

First, it is important to have a feeling for the basic concepts of object-oriented programming such as parent, child, class and inheritance. Specifically, a bounding window is a parent, and subsidiary items within it, such as buttons and listboxes, are its children.

A simple requirement might be to have the user press one of a set of buttons. Relevant `wd` commands must cause first the display of the intended buttons, and secondly the detection of whichever button the user eventually presses. Commands to `wd`, which is just the verb `11!:0`, are passed in the form of character strings in which blanks are delimiters. The command

```
wd 'pc a;'
```

means 'parent create' with title 'a'. All commands to `wd` are terminated by `;`, and several may be included in one string, so that

```
wd 'pc a;pshow;'
```

both creates a parent window and causes it to be displayed. If you can, do this now, and experience what may be your first encounter with the world of windows programming.

Suppose now that an application is to have two buttons marked FRESH and FROZEN, and that the user is to press just one of them. `wd` works in a co-ordinate system in which 4 units in the horizontal direction equals one character width, and 8 units in the vertical direction equals one character height. The next pair of commands are `xywh` (standing for `x` and `y` distances from top left, width, height) and `cc` (child create) which position and label the first button:

```
wd 'xywh 20 10 40 12;cc FRESH button;'
```

Assuming that the second button is to appear vertically below the first, its `wd` command can be consolidated with the others so far in a defined noun (character matrix):

```
push=.0 : 0
pc a;pshow;
xywh 20 10 40 12;cc FRESH button;
xywh 20 30 40 12;cc FROZEN button;
)
```

The verb `wd` can accept a table as an argument, and so

```
wd push      NB. No ';' since push is the argument of verb wd
```

causes the window to appear, ready for user input. To receive and interpret this input, you must use `wait;`. This command passes control to the user, following whose actions an `n` by 2 table is returned. The crucial information, that is the name of the button which was pressed, is in the second column of the third row. Once this is safely stored, say in a global variable `RES`, the window can be erased using the `pclose;` (parent close), or alternatively *all* windows can be closed by `reset;`. The actions described in this paragraph can be consolidated in a verb:

```
runpush=.3 : 0
wd push
RES=:>(<2 1){wd 'wait;' NB.(<2 1) indexes the 2nd column of the 3rd row
wd 'pclose;'
)
```

Type `runpush'`, press the button of your choice, and the result string is stored in RES to await whatever processing follows.

This may not seem very appealing programming, but if you have followed the above successfully you will have mastered the essentials of using `wd` — everything else is a matter of absorbing more detail. The remaining examples use the same principle of defining a noun to set up a window, followed by a verb which applies `wd` to it. After running the window, the verb goes on to extract the required information. The general syntax of the `cc` command is :

```
cc name class style ;
```

There are ten class types of which the one investigated so far is `button`. `Button` has two other variations, `check box` and `radio button`. An appropriate noun/verb pair for two check boxes is:

```
check=.0 : 0
pc a;pshow;
xywh 20 10 40 12;cc b1 button bs_autocheckboxbox;
                                NB. bs_autocheckboxbox is a style
xywh 20 30 40 12;cc b2 button bs_autocheckboxbox;
)

runcheck=.3 : 0
wd check
RES=:4).wd 'wait;'      NB. Results (button names plus 0/1 for nocheck/check) are in
wd 'pclose;'           NB. the final rows (all but the first 4) of the n by 2 table
)
```

Instead of repeating the lengthy parameter in the third line of `check`, it is possible to use `cc1`, standing for `create child and inherit`, that is `inherit` from the last `cc`. Also computer users will be familiar with the way in which `check` buttons can be grouped in an encompassing box. Putting these two ideas together leads to the following revision of `check`:

```
group=.0 : 0
pc a;pshow;
xywh 5 2 60 50;cc bgroup button bs_groupbox;
    NB. bgroup is the name of the group box
xywh 20 10 40 12;cc b1 button bs_autocheckboxbox;
    NB. b1 and b2 are the button names
xywh 20 30 40 12;cc1 b2 group;
    NB. bs_autocheckboxbox is a style
)
```

```

    rungroup=.3 : 0
    wd group
    RES=:4).wd 'wait;'
    wd 'pclose;'
)

```

Radio buttons differ from check boxes only in style. The example below demonstrates another feature, namely setting the initial state of one of the buttons to 'on' using `csel` standing for 'child select':

```

    radios=.0 : 0
    pc a;pshow;
    xywh 20 10 40 12; cc b1 button bs_autoradiobutton;
    xywh 20 30 40 12; cci b2 group; csel b2 ccheck 1;
)

    runradios=.3 : 0
    wd check
    RES=:4).wd 'wait;'
    wd 'reset;'
)

```

The second window class is *listbox*. In principle, just one new command is needed, namely `ctext` which writes strings into the box. However Carriage Returns, which include spaces, are needed to separate list box lines, whereas spaces are already used to separate tokens in the `wd` command string. This is dealt with by bracketing the list box items within instances of `EAV=.255{a.` (the last character in the atomic vector) as shown in the noun/verb pair below:

```

    list=.0 : 0
    pc a;pshow;

    xywh 20 10 40 30;cc list1 listbox vs_vscroll; NB. style is Vertical Scroll
)
    wdctext=.( 'ctext ',EAV)&,@(, &(EAV, ')) NB. Verb to add bracketing EAVs

    runlist=.3 : 0
    t=.;('france';'germany';'italy';'spain'),&. >CR
    wd list NB. List box lines, each followed
    wd wdctext t NB. CR=Carriage Return
    RES=:>(<4 1){wd 'wait;' NB. Use wdctext to build ctext command
    wd 'reset;' NB. Result is in 2nd column of 5th row
)

```

The third class of window is *editbox*. This is similar in principle to *listbox*, except that the result is the (possibly edited) contents of the box. Several style parameters are used.

```

        edbox=.0 : 0
pc a;pshow:
xyvh 20 10 50 70;cc e1 edit es_multiline vs_vscroll vs_hscroll vs_border;
)

        runedbox=.3 : 0
t=. 'france'; 'is an'; 'interesting sort of'; 'country'
vd edbox
vd wdctext ;t.&.>CR
RES=:>(<4 1){vd 'wait;'
vd 'reset;'
)

```

Change, say, est to fer, and the result is the listbox text with this edit carried out.

The fourth class is *combobox* which allows any combination of the previous three to appear within one window. The fifth is *scrollbar* in which the user controls a slider, which may be horizontal or vertical, and the result is its one-dimensional co-ordinate when the user releases the mouse button. The arguments of *ctext* have a different meaning in the context of scrollbar; they are respectively the left position of the slider, the current position, the right position and the number of scaled units per click.

The sixth class is *static* which allows descriptive text to appear within a window using the command *cn* (child name). These last two classes are illustrated in the following noun/verb pair:

```

        sbar=.0 : 0
pc a;pshow:
xyvh 10 10 80 12;cc s1 static;cn "Slider";      NB. Double quotes delimit text
xyvh 10 30 80 12;cc s2 scrollbar;ctext 0 250 1000 25;
                                                NB. See note above on ctext
)

        runsbar=.3 : 0
vd sbar
RES=:4).vd 'wait;'
vd 'pclose;'
)

```

Full details of all the classes and their parameters are given in Help/User Manual/8 Window Controls/Child Classes. Thus equipped, it is a simple matter to move to the next two classes *isigraph* and *isipicture*.

Taking the latter first, the following two examples should be an adequate demonstration:

```
wd 'pc picture; cc p isipicture; ctext j2y.ico;pshow;'
wd 'pc picture; cc p isipicture;
      ctext examples\data\ship.bmp; pshow;'
```

The bitmap ship. bmp is supplied with the J2 system.

Now try some graphs. Two new commands are `grgb` which defines a colour in terms of its red, blue and green constituents, each of which contributes on a scale of 0-255. `gpen` gives the line thickness and style respectively, and `grect` defines the rectangle whose diagonal points are (100,200) and (500,700). The effect of the second line is to display this rectangle in a window co-ordinate system whose ranges are from 0 to 1000 along each axis.

```
graph=.0 : 0
pc graph; cc g isigraph; grgb 200 150 0; gpen 1 ps_dot;
      grect 100 200 500 700;
gshow; pshow;
)

rungraph=.3 : 0
wd graph
wd 'pclose;'
)
```

Now enrich the graph noun by stages, a line at a time. The second line below draws an ellipse within the rectangular framework defined by the co-ordinates. The third changes the drawing colour to pure red, and draws the chord and arc of the ellipse which is cut off by the line joining the points (100,600) and (500,400).

```
graph=.0 : 0
pc graph; cc g isigraph; grgb 200 150 0; gpen 1 ps_dot; grect 100 200 500 700;
grgb 0 200 200; gpen 50; gellipse 120 220 480 680;
grgb 255 0 0; gpen 10; gchord 120 220 480 680 100 600 500 400;
gshow; pshow;
)

rungraph=.3 : 0
wd graph
wd 'pclose;'
)
```

Of course, there is much, much more by way of graphics possibility, which you are now in a position to explore by referring to the `wd` Commands section of the online manual, or by issuing either User Manual/Window Driver, or User Manual WD Reference. To complete the picture, the ninth and tenth classes relate to the more sophisticated areas of OLE and VBX extensions.

Tool bars and their associated drop-down menus are generated in the following way:

```

    menu=.0 : 0
    pco a;pshow;
    menupop "&File";
    menu open "&Open"; menusep;
    menu save "&Save"; menupopz;
    )

    runmenu=.3 : 0
    wd menu
    RES=:>(<2 1){vd 'wait;'
    vd 'reset;'
    )

```

NB. One menupop per Toolbar entry
 NB. Two dropdown items, Open and Save
 NB. menupopz matches menupop

Message boxes are in some ways the easiest windows of all to generate, for example:

```
wd 'mb messi hello mb_ok;'
```

There is no requirement to generate a parent, and mb replaces cc as the command. The cc syntax applies to mb, so that messi above is the name of the message box. Ten essentially different styles are available of which mb_ok is the simplest, that is the message box contains a single button marked OK, pressing which closes the window. There is no requirement for an explicit pclose; or reset;. Five other button combinations are available, namely mb_okcancel, mb_retrycancel, mb_yesno, mb_yesnocancel and mb_abortretryignore. Four possibilities with icons are available, namely mb_iconinformation, mb_iconexclamation, mb_question, and mb_iconstop. An even simpler way to deliver a message is to use the verb wdinfo which is available as a result of running the standard profile. This uses the mb_iconinformation, using the character string right argument to wdinfo.



Renaissance Data Systems
P. O. Box 421 - V
Georgetown, CT 06829
(212) 864-3078

**Books on APL and J and other curiosities
of merit!**

Renaissance Data Systems announces a change in its mailing address. Please note that the telephone number remains the same.

If you would like a copy of our latest **catalog**, please send us a self-addressed legal sized envelope with one first class stamp (if in the U.S.).

Included are such titles as: **APL is EASY**, **APL - An Interactive Approach**, **APL2 at a Glance**, **APL - the Language and its Actuarial Applications**, **APL as a Tool of Thought** proceedings, **I-APL** publications and software, **Boolean Functions and Techniques**, **The FinnAPL Idiom List**, **The Toronto APL Toolkit**, **Mathematical Experiments on the Computer**, **Probability in APL**, **APL - Stat: Do it yourself guide to computational statistics**, **A Source Book in APL** - Approximately 80 titles in all!

We also carry J publications, including **Programming in J**, **An Implementation in J (structure and source code)**, **Arithmetic**, and **Calculus**.

Shareware and commercial APL interpreters and J2 interpreters are available as well.

APL Product Guide

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: 100331.644@Compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 19" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.3Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDDM and the full line of Interprocess APL2 enhancements.
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Atlantis Software	Analytic Platform (K)	poa	K is an APL-like language
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	260	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v1.8	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Any user purchasing APL+Win during 1996 will receive free updates to Vn 1.8 and Vn 2.0 (user to pay carriage)
	Upgrade to Version 1.8	540	From earlier versions of APL+Win. Free update to Version 2.0 (user to pay carriage)

	Migration to APL+Win	620	from APL*PLUS II versions 4/5. Free update to Version 2.0 (user to pay carriage)
		750	from earlier versions of APL*PLUS II
	APL+DOS	1300	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	620 / 390	from APL*PLUS/PC or APL*PLUS II
	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	Strand Software Inc		Strand Software Inc has the sole selling rights to Iverson Software Inc products. I-APL stocks a few of these (mainly APLWIN and the personal J products and books), but is no longer an agent.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG, PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$50	
	J Standard Edition	\$150	

	J Professional	\$325	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$40	
	J Primer	\$40	
	Set of the above 4 books	\$160	
	Concrete Math	\$40	
	Fractals, Visualization & J	\$50	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
	APLWIN	\$70	For 386/PC under Windows 3.1
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis Systems	Dyalog APL	poa	Dyalog Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS II), APL+Unk		

	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	
Uniware	APL+PC	poa	Uniware is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
The APL Group	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler
(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.

Causeway	Causeway for Dyalog/W	35/\$50	Registration for the Causeway platform supplied with Dyalog APL/W as shareware.
	Rain Graphics (shareware)	35/\$50	Annual registration covers full on-line documentation (Windows Help) and handy-reference for the Rain graphics workspace supplied with Dyalog/W.
	RainPro Graphics (professional)	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above)
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support system for top management. Handles large multi-dimensional tables, data analysis, EIS presentations; generates HTML and LaTeX output. Platforms: DOS, APL+II, Windows 3.1/95 for Dyalog APL, LAN support. Ideal for APL customisation, more than 100 installed.
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KS3) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson. Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 5764-092
		\$2500	for CMS, Product Number 5764-011
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(PC Version 2)	\$98	As above for APL*PLUS/PC.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.

Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
Interprocess Systems APL2 Development Workstation			
	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
(mainframe)	AFM	\$15300	High performance component and keyed file system (VS APL and APL2)
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	WSORG	\$1500	Full-screen Workspace Organizer for APL2.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; includes full-screen interface and stand-alone functions. Price depends on number of users.
Lingo Allegro	FRESCO Business Graphics	poa	Fast and Easy Business Graphics DLL
	AP126/PC	poa	GDDM interface for Dyalog APL/W
	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP Interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	TOPR	poa	APL Code and Application Management for Dyalog APL/W
Mercia	LOGOL 92	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +J v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMs together with a sample +Win workspace.
Soliton Associates	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server..
	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data

UNIWARE

APL+Win Monthly Training Program	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
Advanced Windows Programming ...	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
DLL parser for +Win	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
APL+Link Pro	poa	ODBC interface for APL+Win
SQAPL Pro	poa	ODBC interface for Dyalog APL/W
RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
GraphX and ChartFX	poa	High-quality business graphics for APL+Win
Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Warwick University	BATS	250 Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free Training program for the above.
Zark	APL Tutor (PC)	\$299 APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000 Mainframe version.
	Zark ACE	\$99 APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95 488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mt Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	APL Programming	poa	Short or long-term consultant available.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, specialises in tree-processing algorithms.
APL People	Consultancy	poa	Consultants available at all levels. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe

Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
David Crossley	Consultancy	poa	Broad experience in many APL environments
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dogon Research	Consultancy	poa	APL Systems consultancy, design, implementation, support, documentation and maintenance. All dialects with special emphasis on APL2 and Dyalog APL/W.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
Evestic AB	Consultancy	poa	Excellent track record from 10+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
Entropy Software Ltd	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	poa	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL+PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Kestrel	Consultancy	poa	All APLs, all environments. Design, analysis, coding, maintenance, documentation, training, interfacing.
Phil Last	Consultancy	poa	
Lingo Allegro USA	Consultancy	poa	General APL consulting; Migration and Downsizing; Performance Tuning.

Mackay Kinloch Ltd	Consultancy	170-320	Design, analysis and programming for banking, insurance, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consultancy
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis Systems	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows Interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL Interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Uniware	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Wickliffe Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL People	Employment Agency	poa	Employees placed at all levels.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.

	The BBS/APL:	\$24 p.a.	703-528-7617, 1200-14400b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL In Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	42	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Kestrel	Employment	poa	Permanent and contract, home and abroad. From individual placement to supply of complete project teams.
	Software Library	poa	Low-cost software distribution service; call for details.
Oasis Systems	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVS LINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	International	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM50
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	-	Mnl-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Rome/Italy SIG	Roma, Italy			
RusAPL	Moscow, Russia	APL Club	Seminars and Annual Meeting	100,000R (students 20,000)
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Obninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qttly SI-Info		SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: +1 (212) 626-0605 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX, UK. Tel: 01245-496647; Fax: 01245-496646.
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9599 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
Adaytum Software	Douglas Rowley	13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Loxmond, Netherlands. Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	23, The Green, Acomb, YORK YO2 5LL, UK Tel: 01904-792670
APL-385	Adrian Smith	Brock House, Gilling East, York YO6 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: +1 (415) 928-2058 Email: frpp21a@prodigy.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnengeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Club Germany	Oleter Lattemann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63459 Compuserve: 100332,1461
The APL Group Inc	Stuart Sawabin	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS UK. Tel: 01225-462602
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4619 Email: elandau@cals.com
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
AUSCAN Software Ltd	Richard Procter	8 Springmount Ave, Toronto, Ontario M5H 2Y4 Canada Tel: +1-416-651-4037 Email: rjp@interlog.com Web: http://www.interlog.com/~rjp/auscan/
BACUS	Joseph De Kerf	Rooiberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19045, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888
Bloomsbury Software	Peter Day	3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481; Fax: 0171-436 0524; CompuServe: 100010,1467
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730036. email: acamacho@clx.compullink.co.uk Reuternet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874697 Email: 100430.740@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Causeway Graphical Systems Ltd	Adrian Smith, Duncan Pearson	5 The Maltings, Castlegate, MALTON, North Yorks YO17 0DP UK Tel: 01653-696760 Fax: 01653-697719 Compuserve: 100265,1564
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 USA C'serve: 73040,3032
Cinerea AB	Rolf Komernark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
CODEWORK Italia srl	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@inrete.it
ComLog Software	Jeff Pedneau	PO Box 5570, Derwood, MD 20855 USA Tel: +1 (301) 990-7063 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: +33 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: +46 31-45 37 40. Fax: +46 31-45 24 23.
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA UK. Tel: 0171-229 5344

Danish User Group	Per Gjerløv	Email: gjerlov@ibm.net
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX UK. Tel: 01604-780241
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028825 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP UK Tel: 0181-520 6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
Entropy Software Ltd	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY UK Tel: 01442-878065 Email: 100412,1305@compuserve.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS UK. Tel/fax: 0181-864 9537
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Maildrop F465, Springfield, MA 01111 USA Tel: +1 (413) 788-8411x2838
H.M.W.Trading Systems Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020,2632@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	5 Thorsden Court, Guildford Road, Woking, Surrey, GU22 7QS UK Tel: 01483-771028 Email: bob@khamsin.demon.co.uk
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Email: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilberston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998 Email: 101740,1203@compuserve.com
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-527190. Compuserve: 100021,3073
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-625181
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (=2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD UK Tel: 01732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2873 Fax: +7 812 311-2184 Email: aim@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: +45 49 76 20 20 Fax: +45 49 76 20 30 Email: info@insight.dk
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 0171-265 1120
Interprocess Systems Inc.	Stella Chamberlain	11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: +1 (404) 410-1700. Fax: +1 (404) 410-1773 Cserve: 70373,2676
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
J Austria	Joachim Hoffmann	Münzgrabenstr. 68, A-8010 Graz, Austria. Tel: +43 (0)316 814529 Fax: +43 (0)316 816683 Email: JoHo@magnet.at
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessemer Drive, Stevenage, Herts SG1 2DX UK. Tel: 01438-310155 Fax: 01438-310131
Phil Last	Phil Last	145 Crossbrook Street, Cheshunt, Herts, EN8 8JY UK. Tel: 01992 633807

Lingo Allegro USA Inc.	Victoria H. Fil	113 McHenry Road, Suite 161, Buffalo Grove, IL 60089 USA Tel: +1 (708) 459-7529 Fax: +1 (708) 459-8501 Email: info@lingo.com
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voicemail: 01426 98 3858 Email: 100010.33@compuserve.com
MasterWork Software Ltd	Fraser Jackson	PO Box 58-036, Tawa, Wellington, New Zealand. Tel and Fax: +64 (4) 232-4440 Email: 100242.2535@compuserve.com
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3185, Australia Tel: +61 3 586 7574 Fax: +61 3 586 7600 Email: hld@dar.cstro.au
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8866 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE UK. Tel: 01730-263843 Email: Ellis@mrtilf.demon.co.uk
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Oasis Systems B.V.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: oasisbv@pl.net or zwart@oasis.nl
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE UK Tel: 0181-763 2490 Fax: 0181-763 2491 Email: 100551.1401@compuserve.com
Potomac APL SIG	John Martin	51 Monroe Street, Plaza East Two, Rockville MA 02050-2421 USA Tel: +1 (301) 762-9372 Fax: +1 (301) 762-9375 Email: jam@acm.org
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB UK. Tel: 01276-855880 Fax: 01276-855301
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (212) 864-3078
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Rochester APL	Gary Dennis	Sollton Associates, 1100 University Avenue, Rochester, NY 14607 USA Email: gsd@ipsalab.tor.sollton.com
The Rochester Group Inc.	Robert Pullman	50 S.Union St., Rochester NY 14607-1828, USA. Tel: not known. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
RusAPL	Boris Makeev	box 971 (for Makeev), Dmitrovskoe Sh., 2, 127434, Moscow, Russia Tel/fax: +7 95 210-7763 Email: makeev@atom.al.x-atom.net
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30244, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks HP8 4QH UK.
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Claridge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8768666 Fax: 0181 8768660
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eecg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Sollton Associates	Laurie Howard	Sollton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@sollton.com
SovAPL	Alexander Skomorokhov	PO Box 5051, Obninsk-5, Kaluga Region, Russia Tel: +7 (08499) 31463 Email: askom2@kaluga.rossmail.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (512) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rhswain@acm.org
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: +1 (817) 577-0165 Compuserve: 73700,2545 Email: syarus@unicomp.net

SwedAPL	Christer Ulfhjeltn	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServer: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sydney APLUG	Bob Bykerk	Bob Bykerk, 100 Lamrock Ave, Bondi, NSW 2026, Australia Tel: +61 2 9365 2520
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91354, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: rjp@interlog.com www.sigapl.mtnlake.com/sigapl/welcome.html
Uniware	Eric Lescasse	Tour Neptune, Cedex 20, 92086 Paris la Défense 1, France. Tel: +33 (1) 47-78-78-00. Fax: +33 (1) 40-90-04-11 Email: iescasse@uniware.fr
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd, Whitehaven, Cumbria, CA28 6JD UK. Tel: 01946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL UK Tel: 01203-523359
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

WORLD WIDE WEB SITES

ORGANISATION

URL

AFAPL	www.ensmp.fr/~scherer/fanglef/ (Journal available on line)
APL2000	members.aol.com/APL2000
APL-385	www.demon.co.uk/apl385
AUSCAN	www.interlog.com/~rjp/auscan
Capital PC User Group	http://cpoug.org
Causeway	www.causeway.co.uk
CODEWORK	www.codework.de
COSY (Bob Armstrong)	www.cosy.com
Dinosoft Oy	http://yritys.kolumbus.fi/dinosoft
Dyadic Systems Ltd	www.dyadic.com
FinnAPL	http://personal.eunet.fi/pp/apl
Godin London Inc	www.godin.com
IBM APL2	www.torolab.ibm.com/apl/apl2.html
Insight Systems ApS	www.insight.dk
Iverson Software Inc	www.jsoftware.com
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch
MicroAPL Ltd	www.microapl.co.uk
RE Time Tracker Oy	www.rett.fi
Shepp & Associates	www.digitravel.com
SigAPL	www.acm.org/sigapl
Rex Swain	www.pcnnet.com/~rhswain
The APL Group Inc	www.apigroup.com
Toronto SIG	www.sigapl.mtnlake.com/sigapl/welcome.html
Uniware	www.uniware.fr (www.uniware.fr/uk - English)
Jim Weigang	www.chilton.com/~jimw

FTP SITES

ORGANISATION

DOMAIN NAME

IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

REVIEWS SECTION

New Trace Facilities in J3.03

by Roger K.W. Hui

Acknowledgments

I am indebted to Richard Brown and Cliff Reiter for suggestions leading to facilities described herein.

Extension to Error Text

The name of the current verb (if any) is displayed along with the name of the error; having the current name makes it much easier to isolate errors in nested function calls.

Example 0:

```

mean=: sum % #

mean i.9
|value error: sum                the current verb is sum
|      mean i.9

sum=: +/
mean i.9
4

mean 'asdf'
|domain error: sum
|      mean 'asdf'

(+ / % #) 'asdf'
|domain error                the current verb is anonymous
|      (+ / % #) 'asdf'

```

Example 1:

```

mean=: 3 : '(sum y.) % # y.'

mean i.9
|value error: sum
|  (      sum y.)%#y.

sum=: 3 : '+/y.'

mean i.9
4

mean 'asdf'
|domain error: sum
|      +/y.

```

Trace

The conjunction `13!:` controls tracing.

```

u 13!:16 n
(r,c) 13!:16 n

```

The right argument `n` specifies the maximum level of function call to be traced: `0` means no trace; `1` means immediate execution only; `_` means trace everything; etc. The left argument can be a verb to be used for displaying arrays in the trace (and is not itself traced during tracing). It may also be integers `r`, `c`, whence the system default display is used, clipped to `r` rows and `c` columns. (Two numbers suffice as clipping parameters because the output of an `n`-dimensional array is 2-dimensional on the screen.) Finally, it may be the empty vector, whence the current trace level and display controls are shown. The result is `i.0 0`.

The following sets of examples illustrate the use of the trace facility.

0. Controlling Trace
1. Tracing
2. Tacit Verbs
3. Explicit Verbs
4. Operators
5. Errors

Example 0: Controlling Trace

```

trace=: 13!:16
lr   =: 3 : '5!:5<'y.'''      linear display of an array

_ _ trace _      trace everything; display everything
9 trace 1        trace immediate execution only; display maximum of 9 rows
": trace n       same as _ _ trace n
lr trace n       linear display of trace output

```

Example 1: Tracing

Tracing provides information on results *within* a line; the action labels, "0 monad", "1 monad", "9 paren", etc., are from the parse table in Section II E of the *J Introduction and Dictionary*, and reflect the activities of the interpreter with high fidelity.

The commentary on the right is not part of the trace.

```

_ _ (13!:16) _      trace everything; display everything

i. 4
----- 0 monad -----
i.
4
0 1 2 3
=====
0 1 2 3

n=. # 'Cogito, ergo sum.'
----- 0 monad -----
#
Cogito, ergo sum.
17
----- 8 is -----
n
=.
17
=====

(i. 2 4) +/ .* *: 5 * 10 20 30 40
----- 2 dyad -----
5
*
10 20 30 40
50 100 150 200

```

action
verb
argument
result
end of parse
result of immediate exec.

As usual, the result of immediate execution is not displayed if the last action was assignment.

left argument
verb
right argument
result

----- 0 monad -----

i.
2 4
0 1 2 3
4 5 6 7

verb
argument
| result
|

----- 9 paren -----

0 1 2 3
4 5 6 7

----- 3 adverb -----

+
/
+ /

----- 4 conj -----

+ /
.
*
+ / .*

----- 1 monad -----

+:
50 100 150 200
2500 10000 22500 40000

Words to the left of +:
are parsed first because an
operator may "grab" a
verb.

----- 2 dyad -----

0 1 2 3
4 5 6 7
+ / .*
2500 10000 22500 40000
175000 475000

| left argument
|
verb
right argument
result

=====
175000 475000

end of parse
result of immediate exec.

Example 2: Tacit Verbs

sum=. + /
lr (13! : 16) _

linear display of trace output

s=. sum i.9
----- 1 monad -----

i.
9
i.9

linear display of argument
linear display of result

----- 0 monad -----
sum
i.9

+/
i.9

36

trace output is indented
one space for each level
of function call

=====

36

end of trace at this level
result of pending action

----- 8 is -----
s
=,
36
=====

mean=: sum % #

----- 5 trident ----
sum
%

sum % #

----- 8 is -----
mean
=:
sum % #

=====

mean i.9

----- 1 monad -----
i.
9
i.9

----- 0 monad -----
mean
i.9

```
-----
sum % #
1.9
```

```
-----
+ /
1.9
36
```

```
=====
4
```

result of mean

```
=====
4
```

result of pending monad

```
=====
```

4

result of immediate exec.

Example 3: Explicit Verbs

agm defined below computes the arithmetic mean if the left argument is 0 and the geometric mean if it is 1.

```
agm=: 4 : 'if. 0=x. do. (+/ % #)y. else. (*/ %:- #)y.
                                     end.'
```

```
lr (13!:16) _
```

```
0 agm 3 4 5 6
```

```
----- 2 dyad -----
```

```
0
agm
3 4 5 6
```

```
-----
0
4 : 'if. 0=x. do. (+/ % #) y. else. (*/ %:- #) y. end.'
3 4 5 6
```

```
-----
0=x.
```

new line to be parsed

```
----- 2 dyad -----
0
=
0
1

=====
(+/%#)y.                                end of parse
                                         new line to be parsed

----- 3 adverb -----
+
/
+ /

----- 5 trident ----
+ /
%
#
+ / % #

----- 9 paren -----
+ / % #

----- 0 monad -----
+ / % #
3 4 5 6
4.5                                     result of line (+/%#)y.

=====
4.5                                     result of pending dyad

=====
4.5                                     result of line 0 agm ...

=====
4.5                                     result of immediate exec.
```

Example 4: Operators

Tracing an expression involving an operator can provide insight into the internal workings of that operator. Thus:

```
sum=: +/

1r (13!:16) _
```

```

sum"1 i. 3 9

----- 4 conj -----
sum
"
1
sum"1
----- 1 monad -----
i.
3 9
i.3 9
----- 0 monad -----
sum"1
i.3 9
-----
+ /
i. 9
36
=====
+ /
9+i.9
117
=====
+ /
18+i.9
198
=====
36 117 198
=====
36 117 198
sum\1 2 3
----- 3 adverb -----
sum
\
sum\
----- 0 monad -----
sum\
1 2 3
-----
+ /
, 1
1
=====
+ /
1 2
3
=====
+ /
1 2 3
6
=====

```

apply sum to each
vector of i.3 9

cell 0
result of cell 0

cell 1
result of cell 1

cell 2
result of cell 2

overall result assembled
from the individual results

sum prefixes

The verb is applied to
successively longer
prefixes.

```
1 3 6
=====
1 3 6
```

overall result assembled
from individual results

Example 5: Errors

Tracing is quite effective at pinpointing the location of an error, because it shows the events leading up to the error.

```
mean=: sum % #
lr (13!:16) _
mean 1.9
----- 1 monad -----
i.
9
i.9
----- 0 monad -----
mean
i.9
-----
sum % #
i.9
=====
|value error: sum
|   mean 1.9
|   sum=: +/
|----- 3 adverb -----
+
/
+/-
----- 8 is -----
sum
=:
+/-
=====
mean 'asdf'
----- 0 monad -----
mean
'asdf'
-----
sum % #
'asdf'
-----
+/-
'asdf'
=====
|domain error: sum
|   mean'asdf'
```

the current verb is sum

the current verb is sum

RECENT MEETINGS

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

APL2000 User Conference

October 23rd – 24th 1996, Orlando, Florida

reported by Adrian Smith

General Impressions

There was no doubt that this was an overwhelmingly positive conference. I went along not really knowing what to expect; this was the first time that the APL*PLUS (PC and Unix) world had got together after being gradually abandoned by the 'M' company (delegates were willing to speak of Dyadic by name, but not of the other lot!) and I think everyone present was mightily relieved to see their interpreter moving forward again.



The overall attendance was 80+ (excluding APL2000 delegates) which was at least 20 more than expected, so there was some moving of rooms to accommodate meals and sessions. Otherwise the organisation went like clockwork; everyone was given a very nicely indexed folder with all the papers included, an APL2000 tennis shirt, and a name badge with his or her name in **readable** lettering. Future APLnn conferences please note!

The travel was very straightforward, as Orlando is a major holiday destination, and the hotel was comfortable, low-rise, and friendly. It was much more appropriate to this kind of gathering than the vertical kind, being basically a collection of motel-type units (all cars around the outside) clustered around a

large garden area with tennis-courts, 3 swimming pools, and lots of shady areas to sit down in and chat with other delegates. The climate was perfect — hot but not humid during the day and warm enough at night that we could sit out for breakfast at 6.30am watching Venus fade to invisibility as the sun came up. We never did get the 'wake-up' call to work, but the local bird life (of the Mockingbird variety) went off with absolute reliability at 6.30 (and again at 7.00) so this was never a problem!

Conference Diary

Opening Session (Eric Baelen)

The proceedings opened at 8.00 on Wednesday 23rd with breakfast and registration, and Eric Baelen welcomed us to the conference at 9.00 in a typically upbeat style. The delegate list was strongly biased to the USA, with a particular interest from the insurance / healthcare / actuarial sector, but there were representatives of 11 other countries including Finland, France, Sweden, Columbia (maybe a first for an international APL gathering?) and Australia.

Eric's message was a simple one "I'm proud it's in APL" and he encouraged everyone to adopt this as the message of the conference. In fact when any other speaker suggested that some work "looked just like Visual Basic, so we don't need to tell anyone it's in APL" Eric would leap to his feet and admonish them. He introduced delegates to the latest interpreter — a new version was included in every conference pack — and gave a brief roundup of the main sessions to come. It was interesting to see the reaction to the new Gui components, many of which have been available in Dyalog APL for at least 6 months; I would say that the majority of this audience were not even aware of the "D" company at all, and those that had heard of it were totally uninterested. It is probably also true that they do not really need sophisticated Windows programming tools — insurance systems tend to be simple form-filling applications and actuaries are quite happy running their functions from the session prompt. What they need is a fast array engine with a fast, adequate Gui interface: this is exactly what Eric is giving them! What they also need is the ability to do this stuff across corporate networks, which is where the TCP/IP interface comes in — see later!

SoftMed moves to Windows (Chris Lee)

When Chris first came across the SoftMed company, he naturally asked them what they did; on being told "Medical Records" his reaction was the same as everyone else's — well, that should take about 20 minutes, then what do you do? In fact it is a massively complex area, and it is becoming increasingly necessary to get it right. There have been several recent examples of hospitals getting the

wrong record in the wrong place at the wrong time, and people have been hurt or killed as a result. The SoftMed system is the market leader in the US because it includes a very thorough authentication and checking process, and because it was designed by someone who knew what the issues were and how they could be solved by computer.

Chris began by running through the history of the system, then he took us through the somewhat stop-go history of the Windows conversion, before showing us around the current product. It all started 14 years ago when Don Segal saw a need for a software solution, bought an IBM 5120 and programmed a one-hospital system in APL. After 14 years it had been installed in over 1000 hospitals, and consisted of a large collection of APL*PLUS/PC workspaces embodying years of effort making that original little system do all the things the users wanted it to do. It never even moved to PLUS II — many users were working on old 286 kit.

When Windows first came along, SoftMed wondered what to do about it, and got a unanimous reaction from their users: "DOS is faster, simpler ..." and so they quite reasonably stayed with it. Then about 1 year ago the wind suddenly changed. Microsoft started a healthcare consortium, all the new customers wanted Windows and the IS departments started giving Windows 95 machines to many of the existing users. Overnight it became a case of going to Windows or losing market share. When you are the market leader, you get very upset at losing share!

Very early in the design process, SoftMed set some important ground rules:

1. They would protect the existing DOS user base. Bugs would still get fixed and vital enhancements included.
2. They would maintain database compatibility across the old and new systems. This would allow the systems to co-exist and would allow hospitals to upgrade with a simple software installation. One problem with hospital records systems is that you can afford no long periods of downtime, so a massive file-conversion process was never likely to be possible.
3. Don Segal appreciates what the users do — every keystroke counts. If they transcribe 1 less document in a day, they lose money! The system must save keystrokes wherever it can by tabbing well, anticipating input, providing short codes like 't' in date entry fields. Windows is not designed this way!
4. Coding style must be consistent. The old system evolved to conserve symbol table, so it has dense complex code with short meaningless names. In its 12-year lifetime it has seen many APL programmers pass by, some of them not very good. Chris was determined to set some rules on modularity, coding style and code management.

The next phase was what Chris called "Toe-Wetting"; finding a small self-contained project that they could use as a learning experience. They had started work in C on a system to store and retrieve EEG charts; C was used because of the apparent impossibility of building anything useful with the APL*PLUS II Windows interface. They 'sold' the product with a delivery date of February and by the major show in October they were in the state where "... well, it works on my machine" with very little prospect of delivering on time. They couldn't simply abandon it (having already sold it) so they had 4 months to get something off the ground in PLUS III which was becoming stable enough to work with.

Chris' team worked on the 4 or 5 functions that produced the data and they used LEX2000 consultants to develop the Gui front end. By February they had a working product, a satisfied customer and something to demonstrate to the marketplace. Next they tried a simple conversion — ESA or Electronic Signature Authentication. This has a quite complex back end (who can sign for what, under what circumstances) but a very simple interface. It proved a good choice and was moved successfully in January 95.

Having made two successful forays into the Windows world, SoftMed now settled down to draw up the ground rules for a major migration of all the existing systems. They already had a code management system, so one rule was that there should only ever be one live copy of each function. Naming conventions are just a matter of someone setting a rule and everyone sticking to it. As Chris put it "I got to be the somebody, so they're *great* rules". APL does not have static locals, so $\Delta t l a$ means "keep off this variable"; modules have public functions, any others have $\Delta t l a$ (Three Letter Acronym) in front, non-public variables are marked similarly. There is a 'base workspace' which is built by the `]CodeInit` command, and has some 212 common functions and 28 base variables included.

The next big decision was to go for user-defined classes as a way of encapsulating special behaviours. For example they needed a list box which only pulled the data as the user required it, rather than all at once before anything was shown. These were made to feel very like the built-in `QWI` controls which gave a consistent interface to the developer. They also ensured a consistent look and feel to the end user.

The DOS code was riddled with file reads and user prompts, and file writes and prompts. Chris was determined to separate data access code from Gui code — there would be no file reads in Gui code and no dialogue boxes popping up in the data-access code! This immediately meant they were into a significant rewrite, not just a simple code migration.

He was also doing his best to 'raise the bar'; programmers like to solve problems, so let's set them real application problems, not nasty low-level ones. The objective is to build with bigger and bigger pieces, so the application developers can concentrate on putting it all together. An excellent example of this philosophy was a user-defined class which took a matrix of field definitions and added a single control with the fields neatly laid out below each other. It handled scrolling with <Up> and <Down> buttons as required, and had sensible resize behaviour built in. As a result, programmers who were faced with the need to generate many rather simple data-entry forms quickly could stop messing around in *JWED* and simply define the required fields. The class sorted out the spacing, tabbing, and other little details like bolding the caption of the active field. I think of all the things which were shown on the first day, this was the one which really got people's attention! Mental note to put something similar into the next Causeway release!

So much for the groundwork — everything was just nicely ready to start when a stack of urgent work came in and (see objective (1) above) everyone was switched back to the DOS product. Chris did what he could to validate and test the groundwork during this period, and with hindsight the hiatus of nearly a year was time well spent. When the Windows work came back to the front of the queue it did so with a bang, and having a solid base meant that the standards worked, and got used, from day-1.

The two shows that count are in October and February, so when the message went out in July "Windows or Bust — we do not show the DOS product any more" they had to move 3 major apps in a very short time. After a month Chris was behind, as there was more tricky detail in the field-level behaviour than he had anticipated. However he just made it in time, and was sufficiently confident about the February deadline to take time out to give his talk! He was also keen to point out the benefits of working closely with APL2000, who now owned the interpreter and were listening hard to the users (and their own consultants) and fixing the major irritants in the interpreter. The 'conference bag' release has a couple of extra events to ensure that field-exit is always caught, and that it can be rejected if the data is invalid; these are in there because SoftMed was finding it painful to work around all the holes left by validation on UnFocus. Where next? Chris was confident that by February he will have at least one major customer running live with the new system.

FAQs, Tips and Techniques

This was the first of the parallel sessions, and was of most value for the printed material, and the helpful workspaces on the supplied diskette. This included handy code for things like turning the paper around, simulating *EDIT* under

program control, and so on. The questions were mostly oldish chestnuts like 'why this ridiculous co-ordinate system, what's wrong with pixels like everyone else uses?' or legitimate complaints about]WED (which does not handle extensions to styles for existing objects, or any of the recent additions like property sheets). People seemed to be having a lot of trouble with multi-line edit fields, and in general were advised to wait for the RTF edit field, which sounds like a case of sledgehammer and nut. In general, I felt that this session was a good idea which failed to work as well as it should have done — I would be interested to hear what the other attendees thought.

New Language Features (Bill Rutiser)

I was sorry to miss this session; unfortunately it clashed with the first of our two presentations on Rain and NewLeaf. To summarise briefly from the printed notes:

- Ravel with axis, both with integer and fractional axis, giving lamination in the latter case.
- APL2-style partition with axis. Personally I find this quite horrible to use, and bless APL+ for retaining the sensible version as `⌈PENCLOSE`, but I suppose the APL2 compatibility is necessary.
- Axis specification on all the scalar dyadics, so you can add vectors to matrices easily. Fine, and handy, but please can we have rank as an alternative? This is one thing that J does so much more consistently!
- Object attributes with `⌈AT`, again in APL2 style. APL*PLUS has been storing function timestamps since 1984, but only now can you reveal them! You can also check the object's syntax without actually taking the trouble to parse the header. It was noted in the QA session at the end that this is reset whenever the function is fixed, so is of little value to anyone with a file-based code management system. They will have to continue with checking component timestamps until +Win adds Dyalog-style `⌈OR` which preserves the timestamp on fixing.
- N-wise reduction. This is a real gem, and is one of the things that adds genuine 'tool of thought' power to the language. Of course you can do moving averages trivially, but it allows so much more, for example looking for runs in data, or the ultimate one-line 'life' program. This gets a lot more relevant these days when everyone is into bitmap processing — the first thing you do when smoothing jaggies is to count neighbours, and how better to do it than to border with zeros then take the $3+/3+/MAT$? So simple, yet so much in tune with the kind of array-thinking that APL encourages.

All the new features will be in APL+Win and APL+Dos by the end of the year, and APL+Unix early in 1997.

The LEX2000 Financial Reporting Software (Michael Steiner)

This session was a run through the *raison d'être* of APL2000's major application, and a demonstration of several of the more interesting Gui features. The market is defined by the gap between spreadsheets (easy to use, flexible, manually intensive, prone to casual errors, hard to maintain) and General Ledger packages (strong on data integrity, trial balance built in, lots of standard reports, inflexible, limited budgeting and forecasting, lots of redundant accounting structures).

LEX2000 uses the classic 4-dimensional data model (see e.g. Bittlestone, APL Business Technology 83) with dimensions representing:

- **Time.** This is more complex than it sounds, as the accounting periods can vary and it must include moving windows for concepts like 'last quarter'.
- **Account.** Typically this is a consolidated general ledger with a summarised chart of accounts which can be common across geographically separated subsidiaries.
- **Component.** This represents the business organisation, and can be multiple trees, for example by product group, legal company and so on.
- **Submission.** This is an arbitrary label attached to a particular set of accounting data, such as 'Last Year's Budget' or '1995 Actuals'. Typically the system would be asked to make comparisons between submissions.

The demonstration of the finished system showed some neat tricks, for example any three of the four dimensions could be presented as 'Page' 'Row' 'Column' and you chose the presentation simply by dropping the appropriate icon into the correct box. As you clicked and dragged, the icon made itself into a cursor so it looked as if you had picked it up with the mouse. When you dropped it, the report re-arranged automatically.

Support for OCX objects (Patrick Parks)

This wasn't beta code, nor yet alpha code — it was 'aeroplane code' but it all seemed to work and promised a much better integration of the FarPoint grid control, and all the other emerging OCXs, into the user's application. At the moment you need to know the OLE registration string to connect to the object, but soon you will just be able to `OWI 'New' 'FarPoint'` to place a registered control on your form.

Controls will have the obvious properties mapped to the standard `OWI` syntax, so 'where' will work as expected, but will obviously have lots of extra properties which will be passed on untouched. Any event messages will be passed on exactly as they arrive from the OCX, so the co-ordinates on mouse-down could well be xy rather than row-column. Patrick was unsure whether to preface all

control-specific properties with a standard prefix, or just those which could clash with an APL+Win standard, for example if a control had its own 'where' property this would appear in the property list as 'xwhere'.

For the first time, all properties will be case-insensitive ("Maybe it's time to lighten up a little, Patrick ...") 'cos that's how OLE is.

The Knights who say NI (Mark Osborne & Michael Steiner)

This was a very educational two-part session which started with an excellent introduction to the basic ideas in Client-Server, covering everything from simple web browsers, through 'Fat Client' to multi-layer models.

In the first session Michael introduced the basic ideas of Client-Server with the 'coffee shop' analogy. The customer (client) orders "Coffee with cream and sugar" from the bar (server) via the waitress (network). This is a typical 'thin client' example, where the client has nothing to do other than drink the product. Extending the model to 'fat client' would be simple; the customer orders "strong Java" which comes black. He adds sugar and cream to taste, and stirs with the spoon provided. The network does a little more (it carries the spoon), the server does a lot less, and the client takes on the stirring, and can make some choices about the precise dose of sugar and cream required for maximum satisfaction.

Now suppose the order is "Irish coffee with Cheesecake". The barman sends out for the cheesecake and gets an assistant to measure the whiskey. This is typical of a multi-layer Client Server, where the customer sees no difference (other than perhaps an increase in waiting time), but the server may become a client for a while as the request is assigned to other servers, and the results collated and packaged. This is typical of many more advanced Internet applications, where the customer fills in a simple form and hits the <Submit> button.

We were given a very quick demonstration of PC-PC client server, running 'peer to peer', with a neat application implementing a shared whiteboard between two APL sessions. Either side could draw stuff, and have it appear in the other machine, effectively simultaneously. This was run using two video projectors on side-by-side screens, and there was no visible delay — very impressive and all the more so given the total programming time involved: less than 4 hours.

After lunch we continued with a more technical view of the network, and how TCP/IP sockets actually work. Hopefully, *Vector* can talk Mark or Michael into writing up their slides as a proper paper, as I thought they gave a particularly lucid description of the concepts involved and I cannot really do justice to it here.

We started with the **Socket** (a communications endpoint) which has four attributes:

1. **Descriptor.** This is a small integer, assigned by the system when the socket is created and unique within the process that made it.
2. **Host Address.** This is the internet address of the physical machine on which the socket has been created. It looks like 192.168.160.212.
3. **Port Number.** These relate to the service the socket will provide, and several are reserved for standard services, for example 7 (echo), 21 (ftp), 23 (Telnet).
4. **Connection.** A link to a single partner socket.

So, how do we set up a socket and get it to connect to another socket 'out there' on the network? First both client and server make a "Socket" call which creates a socket, allocates a descriptor, and returns it. Now they must fill in items 2 and 3 of the above list, which is done with a "Bind" call, giving the host address and any reasonable free number (such as 2002) for the port. Now the server can be told "Listen" and will be ready to accept incoming connections which have the correct address and port number.

Now the clever bit! The client sends out a "Connect" call from its socket, the server accepts it, clones itself with a new descriptor, and sends an "Accept" thus making the link. The original socket is still there, still listening, and can accept more connections (cloning itself again) as required. The connected sockets can now use "Send" and "Recv" to move arbitrary data across the link, and everything in between is 'just some magic' — we don't care how it happens.

The new system function `⌈ni` handles everything you need to make these sockets, get them talking, and move data between APL systems or any other TCP/IP host which happens to be out there. It will be built into the interpreter on both the PC and Unix platforms, but will be too late to make the 2.0 release. Expect to see it with the first update release in 1997. The general syntax is simply

```
rc+⌈ni 'function' arguments
rc+data ⌈ni 'function' arguments
```

... where the return code is 0 (for OK) followed by some data, or -1 followed by an error number and possibly some additional information. The "Send" and "Recv" functions default to moving data in internal APL format, so you can move arrays very efficiently between processes on different platforms. Other data types such as 'char' and 'int' are provided for communication outside the APL world. Examples of simple function calls could be:

```
⌈ni 'gethostname'
0 delilah
```

... which finds out where we are, or to get the actual address details:

```
□ni 'gethostbyname' 'delilah'
0 delilah 2 4 192.168.160.212
```

... which returns the address type, length and actual value. Now to make a socket:

```
□ni 'socket'
0 3
```

There are arguments, but they default to the common case and can be elided here. The 3 is the descriptor we need (more or less like a file tie number) to set up the remaining socket attributes and start communicating:

```
□ni 'bind' 3 2005 '192.168.160.212'
0 0
```

... so now three of the four attributes are filled in (2005 is our port number) and we can set it listening:

```
□ni 'listen' 3 5 a tie number and queue depth
0 0
```

The queue depth will default to 5 if we leave it out. Now for the fun part! Suppose someone out there has gone through the same sequence and now issues:

```
□ni 'connect' 3 2005 '192.168.160.212'
0 0
```

... giving the port number and address of our server. We can establish the link by accepting the incoming request:

```
□ni 'accept' 3
0 5 2 2005 '192.168.160.216'
```

... getting back a new descriptor for the clone socket thus created, and the address of our new partner. Now we can send and receive in either direction, so the notion of client and server more or less vanishes as soon as the connection is completed:

```
data □ni 'send' 3 'APL'
0 35
```

... the result is the number of bytes transmitted. On the receiver side:

```

[ni 'recv' 5 'APL'
0 there's too many kids in this tub

```

Of course you need to know that something is ready to receive! At the moment the only option is to run a timer and make a 'Select' call to check what is waiting. By the time the product is released, there will be an event (possibly on the system object) to let you know that something has arrived. Finally we must release (untie) the sockets:

```

[ni 'close' 3 4 5 7
0 0

```

... giving a vector of descriptors to be closed.

Of course all you have here is the roadway — it is up to you to build the vehicles to drive down it, and decide how you will load or unload them. This is what is called a 'protocol' and in the simplest case would just be a numeric code appended as the first element of your data to tell the receiver what to do with it. For example in the shared whiteboard application, 3 followed by a 'Draw' command string was interpreted as 'draw this' and so on. If you want to talk to standard applications (such as the *Alta Vista* server or the Netscape client) then you need to look at the messages they exchange and use these protocols yourself. Fortunately this is all character-based and quite simple to copy (Michael included a simple 'spy' version web browser on the demo disk to reveal the underlying message format) so building a basic Web server is much easier than it sounds. As we were well out of time by now, the details were 'left as an exercise for the reader'.

User Defined Classes (Chris Lee)

This was the session I was saddest to miss — again it fell opposite our own slot in the schedule. It was also very squeezed for time, as breakfast over-ran by nearly 20 minutes (it was served out on the terrace and everyone was enjoying it far too much to move inside), but lunch was a fixed point. The basic idea is to extend the built-in classes by adding new properties and methods, and to provide access to all the properties and methods via a cover function *QWI* which worked in exactly the same way as raw *wi* would. (*Query: +Win does not support user-defined events, which Causeway would find it hard to live without. What does Chris do to get around this restriction?*)

Each UDC is defined by a single function, and typically involves the creation of several Gui objects. The newer ones have been constructed from a standard

template which ensures good documentation and structure. Some examples in the handout:

- AAdder which puts up a pair of list boxes and some means of transferring items between them, and sorting them.
- AButtonGrp which takes a vector or array of button values and returns information on the state of the group as a whole.
- ADate which accepts dates as masked input and knows how to handle short code for 'today' or '+n' days from today.
- AFromTo — a pair of date UDCs separated by a 'to' label
- ASpin — an edit field or Date UDC coupled with a pair of spin buttons.

As you can see, the UDCs can be defined in terms of simpler UDCs and so on down to the raw Gui controls. This looks like a very powerful and extensible idea, and probably should be supported by whatever replaces]WED as the standard design tool. More details (and a copy of some of the latest UDCs and their documentation) are available from chris@softmed.com. Chris is happy for anyone to make use of these ideas, and in return only requests that you send him any particularly interesting UDCs that you make.

Roundup

The main conference ended on Thursday, and was followed on Friday by a solid training session for users new to the +Win Gui. Flights being hard to get after Thursday evening, Gill & I skipped before the official finish, but I am sure that the closing session went well. Thanks to all who made it such a useful event, particularly to Sonia Beekman who found the hotel, handled the admin and set up the excellent conference folders. We look forward to meeting them all again next year, same time, same place.

APL Meeting in Denmark

*reported by Gitte Christensen & Stefano Lanzavecchia
(info@insight.dk)*

The Danish APL Special Interest Group under the Danish Data Association hosts a series of late afternoon meetings each season. The content of the meeting varies and includes presentations of new releases of APL interpreters, new add-on products to APL, and Case Studies from APL-users. The attendance varies too, from 20 to 60 APLers.

On November 27th we had invited Bo Grave from the Danish Railroads and Adrian Smith from Causeway.

Bo Grave spoke about his recent work on controlling signals for railroads. His work is based on BeoLogic, a constraint solving system based on binary nested arrays developed by Gert Møller for Bang and Olufsen. If you would like to read more about the background to this, Bo's APL96 paper is available on the web at www.demon.co.uk/apl385/apl96

The system allows the user to define all valid solutions to a given set of input. Having specified all constraints the model is reduced to a dense binary array containing the full operational information to control a complicated system, in Bo's case the railroad signals. One of the big advantages of the system is that since all possible combinations are generated and taken into account the model can be checked for consistency, making sure that any set of input has one and only one valid response from the controlling system. Other uses of the system are not surprisingly the Beolink system by which all musical outlets in your house can be coordinated via the same remote control; also power supply systems can be modelled using Beologic.

Adrian Smith started out by showing the audience some examples of the work Causeway has been doing for SAP using the design tool Causeway. After this introduction a short guided tour of the facilities of Causeway left the audience with the impression that certainly object-oriented style applications were no longer beyond their capabilities as application developers. Causeway is currently shareware being distributed with Dyalog APL but will soon appear in a fully documented Pro version.

The next presentations were the Pro versions of NewLeaf and Rain, which Dyalog APL users have had the opportunity to experiment with for a while.

Now with both products being fully documented and available for both APL+Win and Dyalog APL we should expect to see some flashy output generated from APL applications.

Both utility products are extremely easy to use and generate publication quality output.

APL-users will particularly appreciate the ability of Rain to handle non equidistant axis divisions, a very useful facility when dealing with time series and an area which has not been dealt with properly by the standard graphics DLLs, VBXs and the like.

NewLeaf allows you to lay out, format and design compound reports containing bodytext, tables generated from APL variables, and graphics from Rain in a very flexible and dynamic way. NewLeaf also comes with a layout designer, a facility the audience expressed interest in having available for Rain as well.

At the very end Adrian was asked whether he had a utility for generating Windows help files in an easy way. And you guessed right — of course he had. Having it demonstrated the audience's response was a definite *make this a product too*.

It is always a joy to see ingenious uses of APL and to watch the growing success of commercial APL applications. It is also reassuring to see how tools for Windows APL development now become available as products just like they did for the mainframe environment. It strongly confirms my belief that APL is indeed very much alive and faces the future as a capable tool for implementing simple solutions to complex problems.

Gitte Christensen & Stefano Lanzavecchia
Insight Systems ApS
Nordre Strandvej 119A
DK-3150 Hellebæk
Denmark

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.

Import/Export and the Wealth of Namespaces

by Douglas R. Bohrer

Abstract

This article is a proposal for a mechanism to reference objects in namespaces as local variables which are not erased on exit. The connection of unqualified object names to namespace objects of the same names is defined as a $\square$ function. The connection is localized at the level of the function which makes the connection. The value of the mechanism for object oriented programming is explored. Two advantages of such a scheme, inheritance without duplication and "fire and forget," are discussed. Several other new $\square$ functions and variables are suggested as convenient to the basic proposal.

What's in a Namespace?

Namespaces are a concept recently introduced in Dyalog APL. They are named objects which contain a collection of APL functions and variables. This grouping is self contained, allowing objects with duplicate names to exist in separate namespaces.

Classical APL has only one namespace, the global workspace. Schemes which allowed grouping objects in the past sometimes allowed objects to be packed into packages. However, these objects were packed from and unpacked into the global workspace.

Multiple namespaces behave somewhat like multiple workspaces in that object names can be duplicated. The difference is that multiple namespaces can exist in a single workspace. The root namespace is defined as the classical APL global workspace. Just as the root namespace can contain "children" namespaces, namespaces contained in the root can themselves contain children namespaces.

Namespace Notation

The specific notation for collecting objects into namespaces is not discussed, mainly because I know only what I read in Vector about them. I have had no direct contact with them. I will define "fully qualified" object names as names which show their descent from the root namespace. If an object named *OBJ1* is

contained in a namespace called *NAME1* in the root namespace, then its fully qualified name is *#.NAME1.OBJ1*. By convention, "*#*" is the root namespace.

Importing Namespace Objects

The basic proposal is to define a `IMP` function, `IMP`, which takes a character vector containing the name of a namespace as its right argument. When executed inside a function, `IMP` would localize all names of objects in the namespace. However, instead of making the names local variables with no initial values, `IMP` would instead link all local references to these objects with the namespace objects. Any reference to names of objects in the namespace would use and change the namespace objects. In effect, objects in the namespace would be passed to the function by reference rather than value.

After the function exit, the local link to the namespace objects would be dissolved, returning to the conditions prior to the function call. Ordinary local variables would be erased. Whatever global and local variables were in use in the calling function would be restored to view. However, changes to namespace objects would be permanently retained in the namespace.

For example, if *OBJ1* is the name both of an object in the root and in namespace *NAME1*, then a function call of `IMP '#.NAME1'` would make all subsequent references in the calling function to *OBJ1* a reference to *#.NAME1.OBJ1* and not to *#.OBJ1* in the global workspace.

Under this definition of `IMP`, calling it twice would localize and link the objects in a second namespace. This localization would supersede previous localizations. For example, if *NAME2* is a namespace containing an *OBJ1* of its own, then executing `IMP '#.NAME1'` `IMP '#.NAME2'` would cause references to an unqualified *OBJ1* to refer to *#.NAME2.OBJ1* and instead of either *#.NAME1.OBJ1* or *#.OBJ1*.

As a convenience, I will expand the definition of `IMP` to allow a list of namespaces as a right argument. For example, `IMP '#.NAME1 #.NAME2'` would have the same effect as first doing `IMP '#.NAME1'` and then `IMP '#.NAME2'` in that order.

Sins Visited Upon the Children?

Since namespaces can themselves contain namespaces, it seems logical to define the contained namespaces as children. I'm not sure how to handle inheritance, that is whether objects in the parent namespaces are visible to the children. There

are two alternatives: 1) children inherit all objects from their ancestors or 2) children inherit nothing from their ancestors unless specified explicitly. Let's examine the consequences of each approach.

If child namespaces inherit all objects from their ancestors, then all objects contained in any ancestor namespace would be visible to the child. In this case, the effect of doing `IMP '#.GRANNY.DAD.KID'` would have the same effect as `IMP '#.GRANNY' IMP '#.GRANNY.DAD' IMP '#.GRANNY.DAD.KID'`. The reader should note that inheritance would extend to ALL generations, not merely the biblical 3 or 4 generations. By implication, all the objects in the root namespace would be visible to all other namespaces as long as they had unqualified names distinct from unqualified object names in the child namespaces.

The alternative would be to inherit nothing. This option would mean that doing `IMP '#.GRANNY.DAD.KID'` would make visible only objects in the *KID* namespace. Objects in *GRANNY* and *DAD* would not be visible unless you imported them. To get inheritance from *GRANNY* and *DAD* you would have to import them explicitly: `IMP '#.GRANNY #.GRANNY.DAD #.GRANNY.DAD.KID'`. The implication for the root namespace is that you would have to import it explicitly as well. Root objects perhaps might be defined as visible for functions with no `IMP` calls, but would have to be explicitly imported if `IMP` was called.

Free to Choose

To summarize, inheriting everything has the advantage of requiring less code, but giving less control to the programmer. Inheriting nothing means that you have more control, but have to tediously enumerate all importations at each function level. Traditional APL treatment of local variables as visible to all functions called by the localizing function, which I refer to as global presumption, would tend to argue for inheriting everything. Current fashion in other languages would tend to argue for inheriting nothing automatically.

I think we should choose to inherit everything because it's more consistent with traditional APL treatment of local variables. However, I am not too strong in this conviction. Switching to political mode, I've invented a way to have it both ways, so you are free to choose whichever way you want to go.

Exclusive Locales

I will define a new `⍵` variable, `⍵XLOC`. It will have two allowed values, 0 and 1. If it's 1, all variable references are assumed to be exclusively local. Any global

objects, in the root namespace or anywhere else, have to be explicitly imported, with no automatic inheritance. Any object that's not imported is local exclusively to the function using it. This means that any function you want to use inside another function has to be imported first. Conceptually, this means each function is an island to itself, with a wall around it. The traditional APL global presumption is reversed if $\square XLOC$ is 1. Computer science purists' heaven, where everything has to be declared, or in this case imported, exists when $\square XLOC$ is 1.

The default value of $\square XLOC$ should be 0. If $\square XLOC$ is 0, then traditional APL's global presumption prevails, and inheritance is automatic.

$\square XLOC$ should be localizable and eligible for inclusion in namespaces. Whenever the value of $\square XLOC$ changes, the effect should be applied to the currently executing function. In other words, if you localized $\square XLOC$ and then set it to 1 in function *PURE1*, then all global objects disappear for *PURE1* as well as for functions called by *PURE1*. Importations into *PURE1* would be in the inherit nothing mode. If *PURE1* called function *SLOP1* and *SLOP1* localized $\square XLOC$ and set it to 0, then *SLOP1* would see all objects imported or created by *PURE1*. Imports in *SLOP1* would be in the inherit everything mode. However, if *SLOP1* had no imported namespaces, then *SLOP1* would not see the root namespace or anything created or imported by functions before *PURE1* was called.

If $\square XLOC$ is 1, then it would make sense to allow importation of individual objects as well as whole namespaces. For example, if *OBJ1* is an object in namespace *#.NAME1*, then $\square IMP \ ' \#.NAME1.OBJ1 \ '$ would import just *OBJ1* and nothing else from namespace *NAME1*. On the other hand, if $\square XLOC$ is 0 then it makes no sense to import a specific object. With automatic inheritance, $\square IMP \ ' \#.NAME1.OBJ1 \ '$ would import all of the objects in *NAME1*, as well as all the objects in root.

Using $\square XLOC$, programmers could choose their poison: purist heaven where you call every shot, or sloppy world where any landing you can walk away from is a good landing. They could even switch back and forth to suit their convenience.

What's Your Name, Who's Your Daddy?

If a lot of namespaces are imported, programmers like the author will get confused and need to find out the fully qualified name of any object. I think we will need a $\square WHO$ with a right argument of a list of object names. It could return the fully qualified names of the objects in the list. For example, if *OBJ1* currently refers to an object in namespace *NAME1* and *OBJ2* refers to a root object, then

`□WHO 'OBJ1 OBJ2'` would return `'#.NAME1.OBJ1 #.OBJ2'` as a character result.

Programmers might also want to see which importations have been made at each level of the SI stack. Perhaps `□SINS` could be defined as a function to show the SI stack with imported namespaces, just as `□SINL` shows local variables. Those without `□SINS` could throw the first stones.

Setting a Default Namespace

People being basically lazy, they will want a way to set a default namespace just like they can set a default working file directory. I think `□BNS` should be defined as a localizable variable containing a character vector name of the base namespace. The default value of `□BNS` should be the root namespace `'#'`. The way this would work is as follows. If `□BNS` is `'#.NAME1'` then `□IMP 'NAME2'` would have the same effect as `□IMP '#.NAME1.NAME2'`.

Where Have All My Objects Gone?

One problem yet to be faced is where to put global objects created by functions who have imported namespaces. Should they go to root or to the last imported namespace? I think they should go to the base namespace in `□BNS`. If `□BNS` is `'#.NAME1'` and a previously undefined object `NEW1` is given a value, then since `NEW1` does not have a fully qualified name it is given the fully qualified name of `#.NAME1.NEW1`. Note that globals are created only when `□XLOC` is 0. When `□XLOC` is 1, all variables are assumed to be local.

Note that under the default values for `□XLOC` of 0 and for `□BNS` of `'#'` that the behaviour is exactly the same as classical APL. All references to unqualified global names are references to the root namespace, that is the workspace itself. All global objects created by a function are placed in the root namespace. Unless you create namespaces and import them, or change the values of `□XLOC` or `□BNS`, nothing has changed.

Object Oriented APL

Why go to all this trouble? The main reason to consider it is that the rest of the world has become object oriented. While APL remains a classless language, lacking any definition of object classes, everybody else has gotten very class conscious. Basic now has visible object features and C has improved its grade to C++. Recently, a computer science professor I met socially thought object oriented programming in APL was a really good joke. Face it, the world thinks

we have no class. I think this proposal is a way to be taken more seriously. Specifically, this approach accommodates the encapsulation of objects, of APL functions and variables, required to do object oriented programming. Conceptually it works in a way that's not a violent departure from "classical" APL. The defaults are designed in such a way that classical APL behaviour is unchanged if the new features are not used.

Inheritance Without Duplication

There are major advantages to this approach. The first benefit is inheritance without duplication. Object oriented programming requires inheriting properties from parent objects. Using importation, inheritance can be arranged, without duplication, under function control. Assume that `#.GPBUTTON` is a namespace containing objects for a general purpose button, and `#.SPBUTTON` is a namespace containing only the objects related to a specific button. Then `⊡IMP '#.GPBUTTON #.SPBUTTON'` would give access to all the general purpose button objects and all of the specific button's objects. The namespace `SPBUTTON` would need to contain only objects which were unique to it. All the generic objects in `GPBUTTON` could be shared by every button by importing `GPBUTTON` before importing each specific button's namespace. Further, if objects in `SPBUTTON` had the same names as objects in `GPBUTTON`, all unqualified references would use the specific objects from `SPBUTTON`. Even functions from `GPBUTTON` would use `SPBUTTON` objects when called, as long as the functions had `⊡XLOC` of 0 when called.

In contrast, the scheme explained in Vector seems to allow only one namespace to be active at a time. As a result of this limitation, if you want to have one namespace inherit from another you have to copy the contents from one namespace to another, or you have to refer to the inherited objects using their fully qualified names.

Fire and Forget

The second benefit is "fire and forget," a concept borrowed from military smart bombs. In APL, this would mean that whatever namespaces are used by functions you call, the active namespaces for the calling function are the same after the call as before. The calling function can "fire" functions, which import whatever namespaces and objects they need, and "forget" about what namespaces they use because, on return, imported namespaces are returned to what they were before.

In the scheme for namespaces described in Vector, you can get a fire and forget effect if you call a function in another namespace using the function's fully qualified name. However, using the fully qualified name of the function puts you in its namespace. You have to pass all objects from other namespaces to the function as arguments or refer to the objects inside the function using fully qualified names. If you want your function to change foreign namespace variables and have the name of the namespace as an argument, the function will have to change namespaces or execute strings with the fully qualified names to work.

Let me explain the problem with an example. Suppose `#.GPBUTTON.CLICK` is a function that handles button clicks. It handles button clicks for a lot of specific buttons, including the specific button represented by objects in namespace `#.GPBUTTON.SPBUTTON`. `CLICK` needs to set variables in `SPBUTTON` in response to the click. `CLICK` can't refer to the variables it needs to set using fully qualified names because it's a utility function for a lot of buttons. `CLICK` either can execute strings with a variable containing `'#.GPBUTTON.SPBUTTON.'` put in front of every variable name or it can change the active namespace to `#.GPBUTTON.SPBUTTON`. The executions are ugly and time-consuming, but changing the namespace makes fire and forget a problem.

As I understand the current scheme, namespace activations are a global effect, good forever. If a function changes the active namespace, then the new namespace remains active even after the function that changed it returns. Further, the new namespace remains active until the next explicit change. If a called function, like `CLICK`, needs to change the active namespace, the called function also needs to change the namespace explicitly back to the way it was before it returns. Otherwise, the calling function has to check after each function call to see if the active namespace is still what's expected.

In contrast, the effects of `IMP` are defined as local. Any called function can use whatever namespaces it needs, without having to explicitly clean up after itself. The calling function retains its imported namespaces automatically. Programmers can fire functions and forget about what namespaces they are going to use and change.

Jus' Like the Rich Folks

Another benefit of this proposal is that it answers computer science purists' complaints about the scope of local variables in APL. In most of the languages beloved by the purists, the scope of local variables is strictly the function they are declared in. Purists detest the global presumption of classical APL, that all local

variables declared in a calling function, as well as global variables, are visible as globals to all functions subsequently called. It bothers them almost as much as naked GO TO's. We seem to be in the process of getting rid of GO TO's, so we might as well work on getting around the global presumption as well. Using this mechanism, we can force everything to be explicitly imported, behaving "just like the rich folks" want us to behave. And in the privacy of our own code, we can set `XLLOC` back to 0 and do what we want.

Caveats

You should remember that I have had no direct contact with namespace implementations. I literally know only what I read in Vector in the articles listed below. I may not have understood them correctly, and if so then that's my fault entirely. I have tried to present my scheme as an independent view of how to handle namespaces, hoping you can forgive any mistakes in my comments on the current implementation.

Also, I feel it's fair to tell you that I am not working for any APL language implementor in any capacity, so I can't make them read this, let alone implement it. I do have a lot of practical programming experience in APL, which I first learned in 1968. This mechanism would be extremely useful in work I'm doing right now. If this mechanism were implemented right now, I would be using it.

Douglas R. Bohrer
Data Processing Manager
Consulting Services
EVEREN Securities Inc.

Bibliography

Peter Donnelly, *"The Use of Namespaces for Encapsulation – a Practical Introduction,"* Vector, Vol.11 No.3, January, 1995, pp.66-74.

Kimmo Kekäläinen, *"Namespaces: A Way to a Well Organized World or Just Another Means to Multiply your Chaos,"* Vector, Vol.11 No.3, January, 1995, pp.92-95

Eric Lescasse, *"Namespaces,"* Vector, Vol.11 No.3, January, 1995, pp.75-91

How to Ride a "Bicypher"

by Ray Cannon (100430,740@compuserve.com)

Bicycle, *bi'si-kl*, n, A two-wheeled vehicle propelled by the rider.
(The Graduate English Dictionary.)

Bicypher, *bi'si'fer*, n, A two-keyed form of secret writing.
(The Cannon Warped Mind.)

Working examples of Public Key Cryptography with Small Integers

I have been fascinated by the concept of Public Key Cryptography, that is where a message encoded with a public key can only be decoded with the corresponding private key. Additionally to hiding the contents of a message, the system can be used in reverse to authenticate who the sender is. By the sender encoding a "signature" with their private key, only their public key decodes it, proving it came from them.

I will try to explain the process with 3 simple APL functions and examples using small integers, leaving it up to you to try them with the code John Sullivan provided in his excellent articles on "Multiprecision Arithmetic" [1], that is, if you have the time and a very fast computer. For a more detailed and accurate description see "New Applications of Mathematics" [2], which was my source and inspiration. I have simplified the process somewhat, as I have never fully mastered modulo arithmetic, and still don't know the difference between encoding, enciphering, encyphering and encrypting.

The Functions

I have one function to encode a text string "enc", and a second to decode the result "dec". These handle text-to-numeric and numeric-to-text conversions. Both functions use a utility that I call "modpower" which does the real work.

There is also a set of 3 globals: "private" (the private key), "public" (the public key) and "mod" (the base of the modulus arithmetic).

The Explanation

The whole process relies heavily on some aspects of modular arithmetic, which I won't go into (mainly because I don't understand it). See [2] above.

Try the expressions below with the set $\text{mod}=17$, $\text{public}=5$ and $\text{private}=13$; or an alternative set $\text{mod}=143$, $\text{public}=73$ and $\text{private}=97$.

The key to the way it works is that for certain sets of values of "public", "private" and "mod":

$$1 \quad (110) = (\text{public} \times \text{private}) \bmod \text{power} \quad 110 \quad (\text{Expression 1})$$

Using both the public and private keys, the expression has no visible effect on the message. But if we just use the "public" key we can encipher the message thus:

$$\text{public modpower} \quad 110 \quad (\text{Expression 2})$$

0 1 46 113 114 60 50 106 96 42

(Note that the actual values returned will depend on the values of "mod" and "private".)

Following on, to decipher the message we repeat the process on the last result but this time using the "private" key

$$\text{private modpower} \quad 0 \ 1 \ 46 \ 113 \ 114 \ 60 \ 50 \ 106 \ 96 \ 42 \quad (\text{Expression 3})$$

0 1 2 3 4 5 6 7 8 9

I like to think of this process as a cog wheel, with a ratchet on it so that it only turns in one direction. Each movement of the wheel jumbles up the message (like rotating a face on a Rubic cube) but each "complete rotation" brings it back to the original message (see expression 1 above). The public key turns it through x degrees, where x is less than 360 (see expression 2 above), the private key through $360-x$ degrees (see expression 3 above). The public and private keys complement one another and are equivalent to an integer number of rotations.

Note that after expression 2 you can't turn it back through $-x$ degrees because of the ratchet effect built into modpower. The ratchet effect is partially produced by the effects of modulus arithmetic, which because it loses information, is irreversible. The reverse process produces multiple possible solutions of which only one is correct.

Connecting Public and Private

Without going into the maths of it, when "mod" is prime, the "public" and "private" keys are connected by :

$$1 = (\text{mod}-1) | \text{public} \times \text{private} \quad (\text{Expression 4})$$

or if "mod" is the product of 2 prime numbers p and q :

$$1 = ((p-1) \times q - 1) \mid public \times private \quad (\text{Expression 5})$$

Try expression 4 with: $public+5 \diamond private+13 \diamond mod+17$ and expression 5 with: $p+11 \diamond q+13 \diamond tm+(p-1) \times q - 1 \diamond public+73 \diamond private+97 \diamond mod+p \times q$

Creating the Keys

Fixing the values used for "private" "public" and "mod" is actually far harder than the actual encryption/decryption. I will go through the steps I used to create a public/private key set for the simple example above:

1. Choose two prime numbers p and q , e.g. 11 and 13.
In real life, to make the code extremely difficult to crack, they would be very large numbers with say 256 bits.
2. Multiply p and q to produce the value to use as "mod", e.g. 143 (11×13).
Note that "mod" would be a 516-bit number with only 2 factors p and q .
3. Calculate the product of $(p-1)$ and $(q-1)$ (call it tm), e.g. 120 (10×12)
4. Choose a prime number less than "mod" as our "public" key, e.g. 73.
5. Calculate a corresponding "private" key which satisfies the equation

$$1 = tm \mid public \times private$$

e.g. 97 ($120 \mid 73 \times 97$) is 1

The 3 functions along with the public key and mod can be published. The private key should be kept secret, as should the values of p , q and tm may be destroyed.

If you are a cipher breaker, and you have been given "mod" and "public", to derive "private" you need to know " tm " which is itself derived from the factors of "mod" (p and q). However, it is VERY difficult (i.e. a long slow process) to calculate the factors of a very large number. Remember we created "mod" as a 512-bit product of two primes.

As to deciphering the encoded message by using the public key and reverse engineering the algorithm used by modpower, think of trying to do the inverse of times reduction on a scalar to produce a vector. The inverse process produces many possible solutions of which only one is the correct answer.

Function Listings

```

r←enc text;⍺IO
⍺Converts <text> string as encyphered numeric vector
⍺IO←0
r←public modpower(,⍺16 16⍺AV;text)
r←dec num;⍺IO
⍺Converts numeric vector to deciphered text
⍺IO←0
r←(⍺AV)[16⍺((⍺num)÷2),2)⍺private modpower num]
r←k modpower c;s
⍺Returns mod|c*k
⍺without domain errors if k is large
s←'mod|',(⍺c),'*'
r←⍺((⍺s)*k-1)⍺s),⍺c

```

Notes on the Function "modpower"

I originally wrote "modpower" as 2 operators in Dyalog APL, but have re-written it in a more simplistic style so it can be run under any APL. The original operators are "modtimes" and "code":

```

r←la(m modtimes)ra
⍺operator to perform m|la×ra
⍺operator can be used with reduction (m modtimes)/vector
r←m|la×ra
r←key(mod code)data
⍺operator to Ecipher <data> with <key> using <mod>
r←(mod modtimes)/(⍺data),key)⍺key/data

```

"modtimes" produces the same result as

```
m|×/la,ra
```

but with the added bonus that if *la,ra* is replaced by a longer vector, (say 500 reshape of 2000) it will still work without getting a domain error that raising 2000 to the 500 power would give. Example using "m" of 143 "private" of 97 and "public" of 73.

```

73 (143 code) 120
1 41 16 108 70 84 46 138 113 10 11 12 52 27 119 81 95 57 6 124
97 (143 code) 73 (143 code) 120
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

```

References

- [1] *Multiprecision Arithmetic*, John Sullivan, Vector vol. 12 nos. 1, 2, 3 & 4
- [2] *New Applications of Mathematics*, Ed by Christine Bondi, Penguin Books, 1991

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Clipboarding APL Arrays	Adrian Smith	90
APL+Win and VBXs	Eric Lescasse	92
J and APL Operators	Norman Thomson	112
APL and J (3): Operators Reduce, Scan, Outer Product	Chris Burke	118
At Play with J: Extended Integers	Gene McDonnell	127
Using a Defined Adverb to Trace Execution in Tacit Programs in J	David Alis	136

Hackers' Corner: Clipboarding APL Arrays

by Adrian Smith

Motivation

I spent a good few weeks this autumn making sure that the versions of *Rain* and *NewLeaf* in Dyalog APL and APL+Win were completely compatible. Pasting blocks of APL code from one to the other (in either direction) is no longer a problem, in fact I have taken to editing code in APL+Win even if the final target is Dyalog — the benefits of code-illumination (so easy to spot accidental globals when they show in yellow) outweigh the minor hassle of a copy+paste.

However, there remained one significant barrier. How do you move an APL array across the great divide? I particularly needed to move *NewLeaf* page layout definitions, which are deeply nested matrices such as:

`Use.Disp rpt.toc`

pg	Table of Con...	200 28	597 845	Orient Portrait	
fr	Body	72 72	~144 ~144		
tx	&r	72 32	~144 0		
tx	Table of Con...	198.5 794	~397 0	Font	tib
				Size	24

... so that I could design in either APL and test the layout on the other one. Here is one (very non-technical) solution to the problem.

Solution

For every array, there exists an APL expression which executes to produce it! This expression is a simple character vector, so it should clipboard with no problem at all. When pasted into the 'other' interpreter, all you need to do is to press <Enter> to regenerate the original array — simple really.

```
xx←'Hello, ''world''' (2 3p16)
lfmt xx
('Hello, ''world''')(2 3p1 2 3 4 5 6)
```

```

      *lfmt xx
Hello, 'world'  1 2 3
                4 5 6
      xx=*lfmt xx
1

```

Here is what I hacked out, with very little thought as to the design, more or less on a case by case basis:

```

v flat←lfmt any;OML
[1]  a Take any array and flatten it for clipboarding
[2]  OML←3 o flat←''
[3]  a Does it have serious shape at this level?
[4]  :If 1<ppany
[5]      flat←flat,(ppany),'p'
[6]      any←,any
[7]      :If 0=ppany
[8]          flat←flat,'θ'
[9]      :End
[10] :End
[11]
[12] a Do we need to go deeper?
[13] :If 1<|=any
[14]     :Select +p,any
[15]     :Case 0
[16]         flat←flat,'θ'
[17]     :Case 1
[18]         flat←flat,+,/'(','(lfmt+any),')'
[19]     :Else
[20]         flat←flat,+,/'(','(lfmt"any"),')'
[21]     :EndSelect
[22] :Else
[23]     :If ' '=1+0pany
[24]         any←(1+any=' ')/any
[25]         any←''',any,'''
[26]     :End
[27]     flat←flat,*,any
[28] :End
[29]
[30] a Unfortunately () is not valid APL!!
[31] :If 0=pflat
[32]     flat←'θ'
[33] :End
v

```

This still fails to produce exact matches on deeply nested empty things, but I really don't care enough to bother — perhaps a keen reader would like to give it some rigorous testing and have fun pointing out all the horrid errors!

As a final aside, I recently came across a paper (dated 1976) which proposed control structures very like those used above. I can't help feeling that if we had had these 20 years ago, APL might still be a mainstream language.

APL+Win and VBXs

by Eric Lescasse (lescasse@uniware.fr)

Introduction

If you are an APL*PLUS user, especially an APL+Win user, or more generally if you are interested in APL and Windows programming, we have started a new on-line program called "Monthly APL+Win Training Program" which is available from our Web site. You may visit <http://www.uniware.fr/uk> to learn more about this program.

This program includes a monthly SETUPi.EXE file to download. The Windows SETUPi.EXE installation program installs a 50-page (or more) WinWord document on your computer as well as an APL+Win workspace, full of reusable APL+Win code, and additional utility files. The WinWord documents explain in detail one or more aspects of advanced Windows programming with APL+Win.

The goal of this program is to teach you how to exploit APL+Win fully to write complete and spectacular Windows applications, as well as providing users with a large library of re-usable and easily adaptable APL+Win code in order to save a lot of development time.

Today's chapter is extracted from one of the monthly WinWord documents: it introduces you to the use of VBXs with APL+Win under Windows 95 by showing how to use the Formula One VBX Excel-compatible spreadsheet from APL+Win.

In this chapter we will learn:

- how to use DLLs from APL+Win
- how to use VBXs from APL+Win (under Windows 3.1 and 95)

Formula One is quite probably the best spreadsheet control available on the market: it is simple enough to use and extremely complete. It will help you solve the following problems with APL+Win:

- exchanging data between APL and Excel
- directly exploiting Excel sheets from APL
- printing sophisticated reports with Excel print quality from APL
- using a spreadsheet or grid object in your APL+Win applications

Sample Formula One VBX Application

To give you an idea of the kind of GUI interface Formula One and APL+Win allow, here is a sample application using a Formula One object:

```
'c:\apibook\disk3\sample.xls' FOneVBX ''
```

	A	B	C	D	E	F	G
1			Sales				
2				Jan	Feb	Mar	Total
3		APL+Dns	\$6 500	\$8 150	\$24 000	\$38 650	
4		APL+Unix	\$18 950	\$4 780	\$32 100	\$55 830	
5		APL+Win	\$62 000	\$85 020	\$97 290	\$244 310	
6		APL+Link Pro	\$3 500	\$9 640	\$8 350	\$21 490	
7		Total	\$90 950	\$107 590	\$161 740	\$360 280	
8							

This looks a little like Excel, doesn't it?

Believe it or not, it is an APL+Win application (written in one day) using a Formula One VBX object. We do not have room here to explain all of this application in detail, but you will get the basics.

Formula One VBX allows you to load any Excel 4 spreadsheet, retaining all its formatting and most of its formulas. Once you have loaded the Excel sheet into an APL+Win form containing a Formula One object, you can print this sheet getting Excel quality printout. It is also very easy to load an APL variable (generally a nested matrix) into a Formula One object and from there to save it as an Excel .XLS file, and as easy to retrieve an Excel .XLS file directly into an APL variable.

As you read these lines, APL+Win 2.0 should be shipping with OCX support. OCXs are the successors of VBXs: if you run Windows 95 or NT, you should use the Formula One OCX. It should be very easy to convert the programs and instructions shown in this article from Formula One VBX to Formula One OCX. We will publish an OCX version of this application in our Monthly APL+Win Training Program.

Preparing to use the Formula One VBX with APL+Win

Starting with version 1.8.07, APL+Win supports VBXs under Windows 95. Of course VBX can also be used with APL+Win under Windows 3.1. In order to be able to use a Formula One VBX with APL+Win, using the new VBX interface, you need to change the Config section of your APLW.INI file (found in the Windows directory) so that it contains the 2 following lines:

```
[Config]
...
APXList=AVB1032
VBXList=vtss
```

The first one (APXList=AVB1032) declares that you want to use the new APL+Win VBX interface. The second one declares that you want to use Formula One VBX, called **vtss**.

You also need to copy the following redistributable Formula One files into your C:\WINDOWS\SYSTEM directory:

```
VTSS.VBX
VTSSDLL.DLL
```

You can get these 2 files by purchasing a copy of Formula One VBX, or by subscribing to our "Monthly APL+Win Training Program" (they are included within file SETUP3.EXE).

Using the Formula One VBX from APL+Win

Once you have made the above changes in your APLW.INI file, you may load your version of APL+Win (it needs to be 1.8.07 or later) and look at the names of the available object classes:

```
'#' [wi 'classes'
Button Check Combo Edit Form Frame Imagelist Label List Listview
MDIForm Menu Option Page Picture Printer Progress Scroll
Selector Spinner Status Timer Toolbox Trackbar Tree VBSSEdit
VBSSView
```

As you can see 2 new classes are showing up: *VBSSEdit* and *VBSSView*. These 2 classes are Formula One object classes.

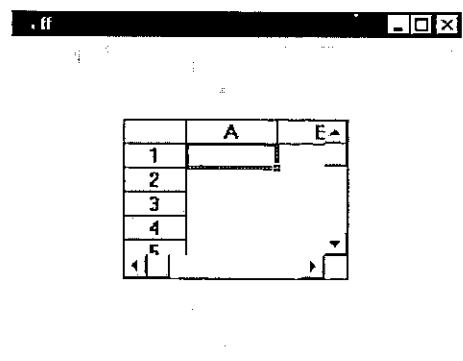
The first one (*VBSSEdit*) is used to get an Edit bar above the grid and the second one (*VBSSView*) is the Formula One spreadsheet object class itself.

The nice thing about third party controls (VBXs and OCXs) is that once you have declared them properly, you can generally use them, just as if they were standard APL+Win controls, using the `WI` system function. In the case of Formula One VBX though, we will also want to use many of the routines included in the `VTSSDLL.DLL` file. More on this later.

Creating a form with a Formula One object

This is as simple as the following:

```
'ff' WI 'New' 'Form' ('scale' 5)
ff
'ff.ss' WI 'New' 'VBSSView'
ff.ss
```

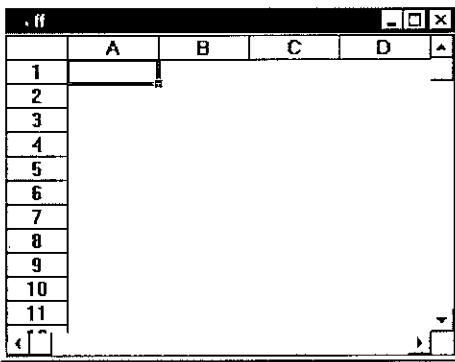


It is advisable to use pixel coordinates (`'scale' 5`) for the APL+Win parent form, since VBXs only understand pixels.

Resizing the Formula One object

One of the first things you will want to do is to resize the Formula One object so that it uses the entire client area of your APL+Win form. This is easily done by changing its `where` property, by:

```
'ff.ss' WI 'where' 0 0, 'ff' WI 'size'
```



However, if you now use your mouse to resize the parent form, the Formula One object does not get resized accordingly to match its new parent form client area. In order to do so, you need to handle the *onResize* event of the parent form.

We will start to write a program with our APL instructions to make things simpler. Here is the first version of our program:

```

▽ FOneEx A;⊞wself
[1]  ⍵ FOneEx A -- Formula One VBX Example
[2]  ⍵ A ↔ ''          a means: build the interface
[3]  ⍵ or 'message'    a means: handle a 'message'
[4]
[5]  :select A
[6]  :case ''
[7]    'ff'⊞wi>Delete'
[8]    ⊞wself+'ff' ⊞wi 'New' 'Form'('scale' 5)'Close'
[9]    ⊞wi'caption' 'Formula One VBX Example'
[10]   ⊞wi'onResize' 'FOneEx'Resize''
[11]
[12]   ⊞wself+'ff.ss' ⊞wi 'New' 'VBSSView'
[13]
[14]   0 0p'ff'⊞wi'Wait'
[15]
[16]  :case 'Resize'
[17]    ⊞wi'.ss.where'0 0,2p⊞warg
[18]
[19]  :end
▽

```

This program should be quite easy to understand.

Its argument is either an empty character vector ('') in which case the interface (the form plus the Formula One object) is built; or it may be a character string message, in which case the program branches to the right *:case* instruction to handle the corresponding message.

This technique is called "callback encapsulation": it allows you to encapsulate ALL callbacks in the very same function which creates the interface. Using this technique, you can have a one-to-one relationship between APL+Win forms and

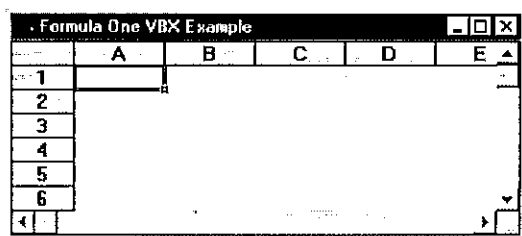
APL+Win functions in your workspace: a form becomes a "unique function" which behaves like a self-contained utility. If you need to use the same form in another workspace, you just need to copy the function (here *FOneEx*) to another workspace and it will still fully run because it will carry with it all its event handling routines.

Coming back to our *FOneEx* function:

- on line 8 we have added a call to the '*Close*' method: this is because we want to avoid seeing the form until the Formula One object has been added to it. By closing the form immediately, it does not show up until a *Wait* method is applied to it (on line 14)
- line 9 sets a caption for the form title bar
- line 10 declares that whenever the user resizes the form, APL+Win should automatically execute: *FOneEx 'Resize'*
- line 14 invokes the *Wait* method: this is where the form gets displayed all at once with the Formula One object on the screen
- line 17 is the instruction we want to be run by the APL+Win system every time the form is resized by the user: it just resizes the Formula One object so that it occupies the whole client area of the form

Let's run this function:

FOneEx ''



Try resizing the form to check our *Resize* callback. It works fine.

Using Formula One Properties

Do not close the above form (or rerun *FOneEx* if you have closed it); press Alt+Tab to go to the APL session. Once an object is alive you can query its list of available properties.

```
'ff.ss' []wi 'properties'
border children class color data def deferexit edge enabled eve
nts extent font help hwnd instance keys methods mode modif
ied modifystop name opened order pointer prompt properties
scale self size state suppress tabgroup tabparent tabstop
tooltip translate vbAllowAppLaunch vbAllowArrows vbAllowD
elete vbAllowEditHeaders vbAllowFillRange vbAllowFormulas
vbAllowInCellEditing vbAllowMoveRange vbAllowObjSelections
...
```

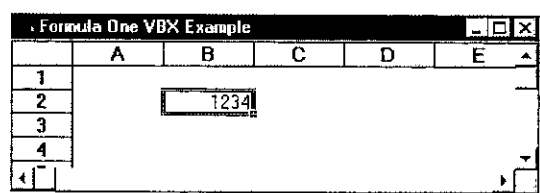
I have not reproduced all of them here because the list is just too long! The properties which do not start with the 'vb' prefix are standard APL+Win properties which still apply to the VBX object (like *color*, *edge*, *enabled*, ..., *where*, ...). All the properties starting with the 'vb' prefix are Formula One specific properties.

Let's try to install a numeric value in the B2 cell. First we need to select the B2 cell. This is done by setting the *vbSelection* Formula One property to 'B2' as follows:

```
'ff.ss' []wi 'vbSelection' 'B2'
```

Press Alt+Tab to look at the Formula One object: cell B2 is now selected. Then we need to clip the numeric value, say 1234, into the selected cell: we can do this by setting the *vbNumber* property as follows:

```
'ff.ss' []wi 'vbNumber' 1234
```



Let's change the number, then try to read this value from the Formula One object by now querying the *vbNumber* property:

```
'ff.ss' []wi 'vbNumber'
```

```
4567
```

```
4567 = 'ff.ss' []wi 'vbNumber'
```

```
1
```

Two other properties can help set the active row and active column in the Formula One object: these are *vbRow* and *vbCol* which are demonstrated in the

next example. So, if we want to fill a complete range with values, we could write a double *for* loop, something like:

```
(L C)+1''pM+5 4p120
:for I :in L
  'ff.ss'wi'vbRow' I
  :for J :in C
    'ff.ss'wi'vbCol' J
    'ff.ss'wi'vbNumber' (M[I;J])
  :end
:end
```

Similarly, if we wanted to read a whole range of values from the Formula One object into an APL variable, we could use:

```
(L C)+1''pM+5 4p0
:for I :in L
  'ff.ss'wi'vbRow' I
  :for J :in C
    'ff.ss'wi'vbCol' J
    M[I;J]+'ff.ss'wi'vbNumber'
  :end
:end
```

However, if we had a large matrix to install into the Formula One object, these techniques would be too slow since they would represent hundreds of lines of interpreted APL code.

Happily, Formula One has other useful properties for writing or reading data into the spreadsheet. The *vbClip* property lets you write a tab-delimited text into a range of cells, all at once. Use it in conjunction with the *DDEFmt* and *DDEUnfmt* utilities which are distributed with APL+Win in the *WINDOWS.W3* workspace. As an example the above loops could be replaced by:

```
'ff.ss'wi'vbClip' (DDEFmt 5 4p120)
```

Formula One VBX Example						
	A	B	C	D	E	F
1	1	2	3	4		
2	5	6	7	8		
3	9	10	11	12		
4	13	14	15	16		
5	17	18	19	20		
6						
7						

and

```

      +'' ❶fi ❷ DDEUnfmt 'ff.ss' ❸wi 'vbClip'
1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16
17 18 19 20

```

which are a little easier to program and really faster to run than the previous loops. Note that *DDEUnfmt* returns a nested matrix of character cells and that *❶fi* is necessary to convert them to numeric.

Formula One has 102 specific properties and it would take much too long to explain all of them here, but once you know how to use one property, you know how to use them all: it's really easy.

However a real spreadsheet object is a complex object which needs many more than 102 properties to be complete and completely customizable. Instead of creating several hundred properties for their spreadsheet Visual Components, the makers of Formula One have decided to write DLL functions. That's why they have created the VTSSDLL.DLL file.

Declaring Formula One DLL Functions

Using DLL functions from APL+Win is a little more difficult than using VBX properties, but once you understand how to do it, it is still pretty simple. The Formula One VTSSDLL.DLL file contains more than 300 DLL functions which you can all use from APL to completely customize the Formula One object.

Almost all of these DLL functions accept one or more arguments, and for almost all of them the first argument is the Formula One object handle.

The Formula One object handle

At any time you may query which is the Formula One object handle by getting its *vbSS* property.

```

      'ff.ss' ❸wi 'vbSS'
1339688375

```

This handle is a long integer which does not change until the Formula One object is closed, so you can read it into a global variable and go on using this variable until the Formula One object is closed.

Rather than using a global variable (which is rarely a good idea), you could write a simple utility called *Hss* which would always return the right Formula One object handle:

```

      v R+Hss
[1]   R+'ff.ss'  wi 'vbss'
      v

```

We will use this function from here on in this article, as the first argument of all our DLL functions.

The APL+Win `⌈na` system function

Before you can use the Formula One DLL functions, you must understand the `⌈na` system function. The `⌈na` system function lets you call any DLL function from a DLL, pass arguments to it and retrieve results from this DLL function. I will only describe the subset of `⌈na` which is necessary to be able to use *VTSSDLL.DLL*. Its general syntax, for 16-bit DLLs (like *VTSSDLL.DLL*), is:

```
'DLL16 res+DLLNAME.fname(arg1,arg2,...,argN)' ⌈na 'fname'
```

where:

- *DLL16* is a fixed required term for a 16-bit DLL
- *res* is a the result type code
- *DLLNAME* is the DLL name which must be in a directory found in the DOS path
- *fname* is the name of the DLL function you want to use
- *arg1*, ... , *argN* are the type codes of the arguments

Note that the result of Formula One DLL functions is almost invariably a return code which you can convert to an explicit error message by using function *SSErrorNumberToText*.

Other rules for using `⌈na` are:

- when an argument is a character string, it must be described as `*C1`
- other types of arguments are:
 - * `I2` short integer (2 bytes, from -32768 to 32767)
 - * `I4` long integer (4 bytes, from -2147483648 to 2147483647)
 - * `F8` floating point (any non integral numeric value)

- when a function needs to retrieve and return some additional information, you need to prefix the argument type with a * (meaning this argument is a pointer, not a value) and to suffix it with + (meaning the argument will be modified and then returned in the result)
- you still need to include parentheses for functions having no arguments

Associating Formula One DLL functions with `¶na`

The only difficulty is in finding the data type to use for each argument. The results, being short integer return codes, are almost always I2.

Concerning the arguments, apply the following simple rules:

- the spreadsheet handle is a long integer and must be I4
- character string is *C1
- if the argument is a character string which must be returned the argument is *C1+
- if you are in any doubt and if you purchased Formula One VBX, you can have a look at the VTSS.TXT distributed file: it contains Visual Basic declarations for all the VTSSDLL.DLL functions. In Visual Basic declarations, the data types are denoted by a one character suffix to the arguments according to the following table:

VB Suffix	Data Type	<code>¶na</code> datatype
%	Short Integer	I2
&	Long Integer	I4
#	Float	F8
@	Currency	F8
\$	String	*C1

The above information should be enough for you to be up and running.

Note that I have designed a product called **DLL Parser for APL+Win** which takes any Visual Basic declaration file like VTSS.TXT and automatically converts it to the exact right set of equivalent `¶na` instructions. For more information on DLL Parser for APL+Win, visit our Web site at <http://www.uniware.fr/dllparse.html>

Examples:

Functions with no arguments:

```
S+ 'DLL16 I2+VTSSDLL.SSClearClipboard()' ¶na 'SSClearClipboard'
S+ 'DLL16 I2+VTSSDLL.SSVersion()' ¶na 'SSVersion'
```

Functions with one argument (the spreadsheet handle):

```
S+'DLL16 I2+VTSSDLL.SSEditCut(I4)' □na 'SSEditCut'
S+'DLL16 I2+VTSSDLL.SSFormatFixed(I4)' □na 'SSFormatFixed'
```

Functions with more arguments:

```
S+'DLL16 I2+VTSSDLL.SSFilePrint(I4,I2)' □na 'SSFilePrint'
S+'DLL16 I2+VTSSDLL.SSInsertRange(I4,I2,I2,I2,I2,I2)' □na 'SSInsertRange'
```

Functions which modify arguments and return them in result:

```
S+'DLL16 I2+VTSSDLL.SSGetActiveCell(I4,*I2+,*I2+)' □na 'SSGetActiveCell'
S+'DLL16 I2+VTSSDLL.SSGetFont(I4,*C1+,*I2+,*I2+,*I2+,*I2+,*I2+,*I4+,*I2+,*I2+)' □na 'SSGetFont'
```

Remarks:

In order to remember Formula One function names more easily you may notice:

- all Formula One functions start with the letters SS
- all Formula One functions that retrieve information start with SSGet
- all Formula One functions that set properties start with SSSet

Using Formula One DLL functions

Now that we know how to declare Formula One DLL functions, we may start to use them. Run the following:

```
FOneEx ''
'ff.ss' □wi 'vbClip' (DDEFmt 6 4p124)
'ff.ss' □wi 'vbSelection' 'B3:C5'
```

Press Alt+Tab: you should see:

Formula One VBX Example						
	A	B	C	D	E	
1	1	2	3	4		
2	5	6	7	8		
3	9	10	11	12		
4	13	14	15	16		
5	17	18	19	20		
6	21	22	23	24		
7						

Now type the following:

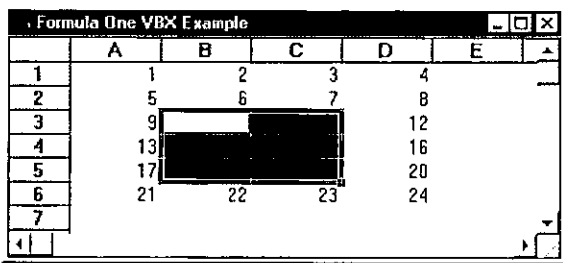
```
'DLL16 I2+VTSSDLL.SSEditCut(I4)'  na 'SSEditCut'
```

if this instruction returns 1, this means the *SSEditCut* Formula One DLL function has been associated in the workspace (and you should see it if you look at the functions in your workspace).

```
SSEditCut Hss
```

0

The 0 return code indicates that the DLL function call has been successful.

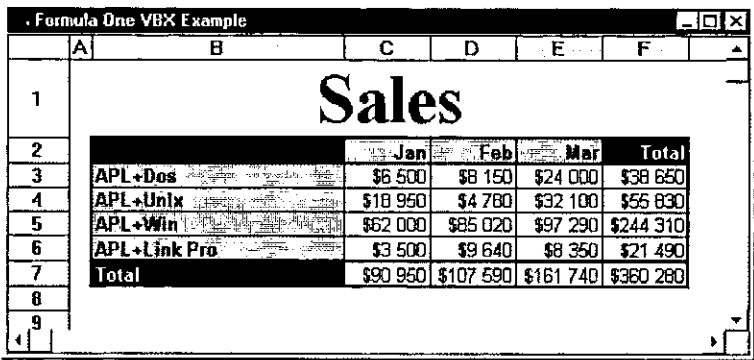


Well, we have successfully used our first Formula One DLL function: with it we could erase a range of cells in the Formula One object, from APL.

Now let's read a whole Excel 4 spreadsheet into our Formula One object. Type:

```
S+'DLL16 I2+VTSSDLL.SSRead(I4,*C1,I2)'  na 'SSRead'  
SSRead Hss 'c:\aplbook\disk3\sample.xls' 2
```

0



	Jan	Feb	Mar	Total
APL+Dos	\$6 500	\$8 150	\$24 000	\$38 650
APL+Unix	\$18 950	\$4 780	\$32 100	\$55 630
APL+Win	\$62 000	\$85 020	\$97 290	\$244 310
APL+Link Pro	\$3 500	\$9 640	\$8 350	\$21 490
Total	\$90 950	\$107 590	\$161 740	\$360 280

The Formula One help file will of course give you full explanations about function arguments. For your information, here is the documentation for SSRead copy pasted from their file.

SSRead Function Call

Description

Reads a worksheet from disk.

Syntax (C)

```
SSERROR SSEXPORTAPI SSRead(
HSS hSS,                // handle to a view
LPCSTR pPathName,       // file path name
int FAR *pFileType      // file type
);
```

Parameters

hSS is a handle to a view.

pPathName is a string containing the name of the file to read. The name can include drive, path, and file name.

pFileType returns the type of file that is read. The following table lists the values returned by this parameter.

Values	Constants	Description
1	kFileFormulaOne	Formula One format
2	kFileExcel4	Excel 4.0 format
3	kFileTabbedText	Tab-delimited text file

The parameter is undefined if SSRead returns an error.

Remarks

SSRead initializes a worksheet structure and reads a worksheet from the specified file. If there is not a worksheet attached to the view, a new worksheet is created.

Let's assume we want to print this spreadsheet now. Type the following:

```
S+'DLL16 I2+VTSSDLL.SSFilePrint(I4,I2)' Ona 'SSFilePrint'
SSFilePrint Hss 0
```

and here we go with a printout of this Sales sheet with the same print quality you would obtain if you had printed this sheet from Excel.

Reading the Excel spreadsheet into an APL nested array

What if we wanted to read this Excel spreadsheet into an APL nested array?

We could use what we learnt so far and a little APL:

```
'ff.ss' ⎕wi 'vbSelection' 'B1:F7'
A+DDEUnfmt Wtoocem 'ff.ss' ⎕wi 'vbClip'
```

A

Sales

	Jan	Feb	Mar	Total
APL+Dos	\$6 500	\$8 150	\$24 000	\$38 650
APL+Unix	\$18 950	\$4 780	\$32 100	\$55 830
APL+Win	\$62 000	\$85 020	\$97 290	\$244 310
APL+Link Pro	\$3 500	\$9 640	\$8 350	\$21 490
Total	\$90 950	\$107 590	\$161 740	\$360 280

```
B+, '$'∘A
(B/,A)+⎕fi''(B/,A)~''∘' $'
```

A find numeric cells
A remove \$, blanks and convert to numeric

A

Sales

	Jan	Feb	Mar	Total
APL+Dos	6500	8150	24000	38650
APL+Unix	18950	4780	32100	55830
APL+Win	62000	85020	97290	244310
APL+Link Pro	3500	9640	8350	21490
Total	90950	107590	161740	360280

And we get a nested APL matrix made of character and numeric cells! Note that the *Wtoocem* function is necessary to convert strings back to APL correctly. You will find it in the distributed *WINDOWS.W3* workspace.

Handling Formula One Events

You can look at the list of available Formula One events with:

```
'ff.ss' ⎕wi 'events'
```

```
Close Destroy Exit ExitError Focus Hide KeyDown KeyPress KeyUp
Modified MouseDown MouseDrag MouseMove MouseUp Move Open P
aint Resize Send Show Unfocus VBCancelEdit VBClick VBDataN
ewRow VBDataRowLoad VBDb1Click VBDragDrop VBDragOver VBEnd
Edit VBEndRecalc VBGotFocus VBKeyDown VBKeyPress VBKeyUp V
BLostFocus VBObjClick VBObjDb1Click VBRClick VBRDb1Click V
BSelChange VBStartEdit VBStartRecalc VBTopLeftChanged
```

Note that all specific Formula One events start with VB.

Let's just handle one specific event: the *VBSelChange* event which occurs each time the current Formula One object selection changes and therefore each time you move to a new cell in the spreadsheet. We want to display the current cell coordinates in 2 status bar fields, as the user moves around the spreadsheet.

Let's create a status bar (a new APL+Win object) in our form.

Change the *FOneEx* function so that it looks like the following:

```

▽ FOneEx A;B;□wself
[1]  aV FOneEx A -- Formula One VBX Example
[2]  aV A ↔ ''          a means: build the interface
[3]  aV or 'message'    a means: handle a 'message'
[4]
[5]  :select A
[6]  :case ''
[7]    'ff'□wi'Delete'
[8]    □wself+'ff' □wi 'New' 'Form'('scale' 5)'Close'
[9]    □wi'caption' 'Formula One VBX Example'
[10]   □wi'onResize' 'FOneEx' 'Resize''
[11]
[12]   □wself+'ff.ss' □wi 'New' 'VBSSView'
[13]   □wi'onVBSELChange' 'FOneEx' 'ss.VBSELChange''
[14]
[15]   □wself+'ff.st' □wi 'New' 'Status'('scale' 5)
[16]     B+1 4p1 'Ready'    0 8
[17]     B+B  2 ' '        20 1
[18]     B+B  3 ' '        20 1
[19]     B+B  1 ' '        0 8
[20]     □wi'status' B
[21]
[22]     0 0p'ff'□wi'Wait'
[23]
[24]   :case 'Resize'
[25]     □wi'.ss.where'0 0,2p□warg
[26]
[27]   :case 'ss.VBSELChange'
[28]     'ff.st'□wi'SetStatus' 2 (▼□wi'vbRow')
[29]     'ff.st'□wi'SetStatus' 3 (▼□wi'vbCol')
[30]   a Alternative using the SSGetActiveCell DLL function:
[31]   a S+ 'DLL16 I2+VTSSDLL.SSGetActiveCell(I4,*I2+,*I2+)' ...
[32]   a (B C)+*''1+SSGetActiveCell ('ff.ss'□wi'vbSS') 0 0
[33]   a 'ff.st'□wi'SetStatus' 2 B
[34]   a 'ff.st'□wi'SetStatus' 3 C
[35]
[36] :end
▽

```

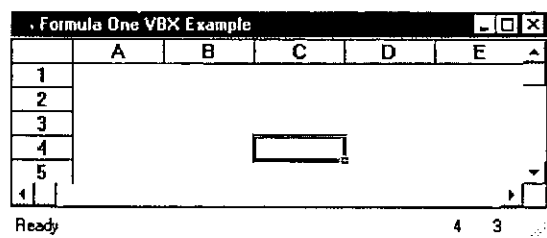
The *FOneEx* function is almost self explanatory.

We have added line 13 to handle the *onVBSElChange* event.

Then we have created a new status object with 4 fields, fields 2 and 3 having 20 pixels width.

The new *VBSElChange* callback is using the Status object method *SetStatus* to fill fields 2 and 3 with the appropriate cell co-ordinates (row and column) as the user moves around in the spreadsheet.

Here is the result:



Unfortunately, I have to stop here: I must dramatically cut the *Monthly APL+Win Training Document* on APL+Win and VBXs, for the sake of available space in *Vector*, but I hope this article will still help many users to start using Formula One from APL+Win.

Just for your curiosity, here is the code of the main function which creates the first form I displayed in this article:

```

V R+A FOneVBX B;C;D;E;F;G;H;I;S;hss;[]wself
[1]  NV A FOneVBX B -- Uses Formula One VBX to access Excel 4 sheets
[2]  NV A ↔ (optional) data to be loaded into spreadsheet
[3]  NV Note: if A is used it must be a nested matrix, each cell
[4]  NV being a character string or a numeric scalar.
[5]  NV If A is a simple character vector it should represent an Excel 4
[6]  NV .XLS file name to load into the spreadsheet.
[7]  NV B ↔ ''          meaning: build the interface
[8]  NV or 'message'    meaning: handle this message
[9]  NV Note: use FOneOCK from APL+Win 2.0 to access Excel 5 & 7 sheets
[10] NV Requires: (F) DDEFmt FormName INIFile InstallDir FileMenu_FOneVBX
[11] NV          ... StatusBar_FOneVBX ToolBar_FOneVBX
[12] NV          ssError ssGet
[13] NV          (Files) BUTTONS.BMP VTSSDLL.DLL VTSS.VBX
[14] NV Copyright (c) 1996 Eric Lescasse [2dec96]
[15]
[16] :select B
[17] :case ''
[18]   :if 0=[unc'A' o A+0 Op] ' o :end
[19]   C+'fmFOne'

```

```

[20]      E+InstallDir
[21]      C [wi'Delete'
[22]      [wself+C [wi'New' 'Form'('scale'5)(\visible'3)'Close'
[23]          [wi'caption' 'Formula One (VBX)'
[24]          [wi'font' 'MS Sans Serif' 8
[25]          [wi'onClose' 'FOneVBX' 'Close'
[26]          [wi'onFocus' 'FOneVBX' 'Focus'
[27]          [wi'onOpen' 'A FOneVBX' 'Open'
[28]          [wi'onResize' 'FOneVBX' 'Resize'
[29]          [wi'onWait' 'FOneVBX' 'Wait'
[30]
[31]      C ToolBar_FOneVBX 1 2 18 0 4 0 5 6 7 0 11 12 0 40 0 36 37 0 1003 0 8
[32]      C ToolBar_FOneVBX 1001 0 1002 0 22 29 45 0 31 25 34 0 92 95
[33]      C FileMenu_FOneVBX ''
[34]      C EditMenu_FOneVBX ''
[35]      C ViewMenu_FOneVBX ''
[36]      C InsertMenu_FOneVBX ''
[37]      C FormatMenu_FOneVBX ''
[38]      C ToolsMenu_FOneVBX ''
[39]      C HelpMenu_FOneVBX ''
[40]      C StatusBar_FOneVBX ''
[41]
[42]      [wself+(C,'.ss')[wi'New' 'VBSSView'
[43]          [wi'vbBorderStyle' 0
[44]          [wi'onVBEndEdit' 'FOneVBX' 'ss.VBEndEdit'
[45]          [wi'onVBSelChange' 'FOneVBX' 'ss.VBSelChange'
[46]          [wi'onVBStartEdit' 'FOneVBX' 'ss.VBStartEdit'
[47]
[48]      R+C [wi'Wait' A
[49]
[50]      :case 'Close'
[51]          C+[wself
[52]          [wself+({~1+C1,'')+C},'ss'
[53]          hss+[wi'vbSS'
[54]          D+~1 ssGet'
[55]          C [wi'value'D
[56]
[57]      :case 'Focus'
[58]          [wi'.ss.Focus'
[59]
[60]      :case 'Open'
[61]          A Initialize VBX object
[62]          hss+[wi'.ss.vbSS'
[63]          S+SSSetHdrSelection hss 1 1 1
[64]          S+SSSetFont hss ('MS Sans Serif',[ctncl]) 8 1 0 0 0 0 0 0
[65]          S+SSSetHdrSelection hss 0 0 0
[66]          S+SSSetSelection hss ~1 ~1 ~1 ~1
[67]          S+SSSetDefaultFont hss ('MS Sans Serif',[ctncl]) 8
[68]          S+SSSetAlignment hss 1 0 2 0
[69]          S+SSSetSelection hss 1 1 1 1
[70]
[71]      :case 'Resize'
[72]          (C D)+2p[wary
[73]          :if 0*p[wi'.fTb1.self'
[74]              [wi'.fTb1.where' (E+0) ~3 32 (D+4)
[75]          :if 0*p[wi'.fTb2.self'

```

```

[76]          [w1'.fTb2.where' (E+30) ^3 32 (D+4)
[77]          :if 0<p[w1'.fTb3.self'
[78]          [w1'.fTb3.where' (E+60) ^3 32 (D+4)
[79]          :if 0<p[w1'.fTb4.self'
[80]          [w1'.fTb4.where' (E+90) ^3 32 (D+4)
[81]          :end
[82]          :end
[83]          :end
[84]          :end
[85]          [w1'.ss.where' (E+30) 0 (C-E+30) D
[86]
[87] :case 'Wait'
[88]   * Install data passed as left argument
[89]   hss=[w1'.ss.vbSS'
[90]   :select +ppA+[warg
[91]   :case 1
[92]     :if 82=[dr A
[93]       S+SSRead hss (A,[tcnul) 2
[94]     :end
[95]   :case 2
[96]     (B C)+pA
[97]     S+SSSetRepaint hss 0
[98]     S+SSSetTabbedText hss 1 1 B C 1 ((DDEFmt A),2p[tcnul)
[99]     S+SSSetColWidthAuto hss 1 1 B C 1
[100]    S+SSSetRowHeightAuto hss 1 1 B C 1
[101]    S+SSSetRepaint hss 1
[102]  :end
[103]
[104] :case 'ss.VBEndEdit'
[105]   :if-([w1'.Atext')=1>[warg
[106]     [w1'.modified'1
[107]   :end
[108]
[109] :case 'ss.VBSelChange'
[110]   hss+[w1'.vbSS'
[111]   (B C)+v''ssError SSGetActiveCell hss 0 0
[112]   D+FormName
[113]   (D, '.st1') [w1'.SetStatus' 5 B
[114]   (D, '.st1') [w1'.SetStatus' 6 C
[115]
[116] :case 'ss.VBStartEdit'
[117]   hss+[w1'.vbSS'
[118]   S+2>SSGetText hss (256p[tcnul) 256
[119]   S+('~1+S[tcnul)+S
[120]   [w1'.Atext'S
[121]
[122] :end

```

v

ToolBar_FoneVBX, StatusBar_FoneVBX and the menu functions (FileMenu_FoneVBX, EditMenu_FoneVBX, ...) have been created in minutes from re-usable template functions called ToolBar, StatusBar, FileMenu, EditMenu, ... which are the kind of reusable APL code we are including in our on-line Monthly APL+Win Training Program.

Conclusion

I would have liked to reproduce the complete article about APL+Win and VBXs in this issue of Vector, but I would need at least 50 pages and of course this was not possible. The complete article goes much into much greater depth concerning Formula One and APL.

We have just touched on the basics of using Formula One VBX with APL+Win here. However I feel you can guess, from these few pages, the tremendous power you can gain from using VBXs like Formula One (and now OCXs as you read these lines) with APL+Win.

With our "Monthly APL+Win Training Program" you will be able to download each month:

- a complete WinWord document (at least 50 pages) talking in depth about one aspect of APL+Win programming
- an APL+Win workspace and accompanying files, full of reusable APL code

If you want to subscribe to the "Monthly APL+Win Training Program", go online to:

<http://www.uniware.fr/uk/subsform.html>

You can download a free trial month from our Web site by entering:

<http://www.uniware.fr/aplbook/setup1.exe>

Uniware
Tour Neptune
92086 PARIS LA DEFENSE CEDEX
tel: +33.1.47.78.78.00
fax: +33.1.40.90.04.11
Email: lescasse@uniware.fr
Web: <http://www.uniware.fr/uk>

J and APL Operators

by Norman Thomson

This article is a mirror image of "APL Operators and J" [1], the thrust of which was that studying J is a useful exercise in sharpening APL programming skills. This article examines the reverse side of this coin, namely ways in which aspects of APL programming can enhance J programming skills. Until now it has been the case that most users came to J from a knowledge base in APL, but it must be expected that there will be an increasing number of new J users who have no acquaintance with APL.

J allows the definition of verbs, adverbs and conjunctions, and the distinctions between these can be confusing. In J, only verbs are executable, just as in APL operators lead to defined functions which are the ultimate executable objects. In J, both adverbs and conjunctions are means of defining verbs. They are distinguished by the fact that adverbs can have one and only one parameter, whereas the meaning of "conjoin" dictates that conjunctions must have two parameters. These may be any combination of noun and verb, and so conjunctions can be classified as nn, nv, vn and vv. This is best illustrated by an example. Suppose that the following pair of verbs is defined:

```
fn=.+. : NB. fn is the verb "square"
vb0=.3 : '2+fn y.' NB. vb0 adds 2 to the square of its
argument
vb0 4
18
```

The notation `vb0=.3 :` is akin to APL `del` in the function header, and allows explicit definition of verbs (that is definitions with explicit reference to arguments), as opposed to tacit programming. An equivalent tacit definition of `vb0` is

```
vb0=.+&2@fn
```

An advantage of defining `vb0` in this way is that it is possible to parameterise the verb `fn` by writing an adverb

```
ad0=.1 : '+&2@x.'
fn ad0 4
18
```

1 : can be thought of as an adverbial equivalent of del, and the rest of the line as resembling the way in which, in the days before APL2 and user-defined operators, APL programmers built up executable lines using character strings, and then prefixed them with "execute". One merit of tacit definition is that it facilitates verb parameterisation through the construction of adverbs.

The 2 in vb0 could also be parameterised. Since there are now two parameters a conjunction must be employed. Define

```

con0=.2 : '+&x.@y.'
2 con0 fn 4
18
9 con0 %: 4
11

```

Again the 2 : plays the role of APL del. : is itself an nn conjunction, joining an integer x. (1=adverb, 2=conjunction, 3=verb) and a character string definition y. . In APL terms, : is a realisation of del which makes a distinction between the part of speech which is being defined.

As a second example, here is a verb which uses control structures:

```

vb1=.3 : 'if. y.<0           NB. returns squares of positive
      do.   r=.%fn y.       NB. values, or reciprocals of
      else. r=.fn y.        NB. squares of negative values
      end.'
vb1 3
9
vb1 _3
0.1111111

```

To parameterise the verb, first obtain a tacit form using gerund:

```
vb1=.fn ` (%@fn) @. (0&>)
```

and define the corresponding adverb

```

ad1=.1 : 'x. ` (%@x.) @. (0&>)'
fn ad1 3
9

```

Recursive functions can also be handled using this technique, for example

```

gn=.*           NB. gn is a verb for parameterisation
vb2=.1: ` ([ gn $:0<:)) @. *       NB. evaluate n.(n-1). ... 1
vb2 4

```

24

```

ad2=.1 : '1: ' (] x. $:0<:) 0. *' NB. define the corresponding adverb
* ad2 4                      NB. product of first 4 +ve integers

24
+ ad2 4                      NB. 1+sum of first 4 +ve integers

11

```

Given the last result, it might be desirable to parameterise also the recursion stopping verb, which suggests a conjunction:

```

con2=.2 : 'y. ' (] x. $:0<:) 0. *' NB. define a corresponding conjunction

+con2 0: 4

10

```

J contains many more primitive conjunctions than adverbs, and a table of both can be helpful in seeking examples of the various possibilities.

Adverbs: / insert /. oblique/key \ train f. fix
 } amend - commute/reflex b. boolean

Conjunctions: nn nv vn vv

&		x	x	x	bond/compose
⌈.	⌈:			x	under/appose
@	@:			x	atop/at
.	..			x	dot prod, even/odd
!	!:			x	obverse, adverse
^	^:		x	x	power
"	"	x	x	x	constant/rank
:	:	x	x	x	explicit defn/tie
@.	@.	x			agenda
D.	D.		x		derivative
!.	!.		x		cut
].	].	x		x	dex
[.	[.		x	x	lev
!	!		x		fit(customise)
~:	~:	x			evoke gerund
!:	!:	x			foreign conjn.

The defined examples given so far, although trivial in themselves, are useful in developing an understanding of parts of speech in J. Here, in conclusion, is a non-trivial example of translating an APL2 operator into a J equivalent. The problem in question is to write a numerical integrator of reasonable generality, the APL2 prototype for which is a pair of operators described in [2].

The starting point is Simpson's rule, for which an APL2 operator is:

```

[0] Z←L(P SIMPSON)R;T
[1] Z←(T×+/(1,((L-1)ρ4 2),1)×P" (↑R)+(T←-R.-÷L)×0,ιL)÷3

```

This realisation of Simpson's rule combines a function P, together with data which is the append of R, the range of integration, and L, the number of intervals, which must be an even number. Call this aggregate an iplan (integration plan), so that the requirement in J is for something to link the integrand (verb) and an iplan. Since we are joining a verb and a vector, the first idea might be to write a vn conjunction. However the iplan is the entire data, and consideration of the vn conjunctions such as power and rank shows that this type of conjunction is appropriate only when the noun conjunct is part of the data. Put in another way, a conjunction is not in itself executable, and so some of the data has to be "held back" for the point at which a verb ultimately achieves control. It is therefore an adverb which must be sought to describe how to execute the integrand, namely at the points of the grid which the Simpson technique calls for. A grid idiom (gridiom!) derived from Chris Burke's admirable J Phrase Book [3] distributed with J Release 3 is the following:

```
grid=+`(*i.)//      NB. y.=left-value, grid-unit, # of points
grid _5 2 5
_5 _3 _1 1 3
```

To turn an iplan into a grid it is necessary first to compute the Simpson sub-interval:

```
subint=.%(%`-/@!.)  NB. sub-interval for iplan=range, # of ints
subint 3 4 4
0.25
```

and to be able to identify the number of intervals, which is one less than the number of grid points:

```
last=.{.@|.      NB. last item in vector as scalar
```

This is all that is necessary to convert an iplan into a grid, using an adverb to describe how (that is, for what values) the integrand is to be applied:

```
itogly=.@{grid@{i.`subint`(>:@last)/.)} NB. by turning iplan into grid

*: itogly 3 4 4
9 10.5625 12.25 14.0625 15
```

Next the Simpson weights have to be calculated. There are several ingenious ways of doing this, but the following algorithm is as comprehensible as any:

```
svts=.3 : '1,((-1+1{y.}$4 2),1'  NB. Simpson weights for iplan

svts 3 4 4
1 4 2 4 1
```

Next these have to be multiplied by one-third of the Simpson interval:

```
swt=.%430(subint * swts)      NB. Adjusts swts for 1/3.h
swt 3 4 4
0.08333333 0.3333333 0.166667 0.333333 0.08333333
```

and finally here is the Simpson adverb which expresses the fact that the Simpson estimate is the inner product of swt and the function values at the appropriate grid points:

```
simp=.1 : 'swt +/ .*x.itogly'  NB. Simpson adverb, e.g. fn simp 0 1 4

*: simp 3 4 4                  NB. Simpson's rule is exact for this fn.
12.3333
```

The APL2 operator ADAPT achieves adaptive integration, which is a powerful generalisation of SIMPSON.

```
[0] Z←EPS(P ADAPT)R
[1] Z←4(P SIMPSON)R
[2] →(EPS>|Z-2(P SIMPSON)R)/0
[3] Z←+/(0.5×EPS)(P ADAPT)"0 1ϕ"R,"0.5×+/R
```

The idea is that the values obtained by applying the Simpson rule for two and four sub-intervals are compared. If these are within a tolerance, which now replaces the number of intervals as the final item of the iplan, then the four-sub-interval value is accepted. If this is not the case, half the tolerance is allocated to each half of the range, and the process is repeated for each half-range. The process of range sub-division continues until all the sub-ranges satisfy the above condition. A first step, then, is to write an adverb simd, which modifies an integrand by describing how to compute the difference between its two- and four-interval Simpson estimates:

```
simd=.1 : 'x.sim2-x.sim4'      NB. Difference between 2 and 4 subints.
sim2=.1 : 'x.simp@,2'          NB. Simpson rule with 2 sub-intervals
sim4=.1 : 'x.simp@,4'          NB. Simpson rule with 4 sub-intervals
```

Next write a verb which generates the new iplans when tolerance is not met:

```
nip=.(hr@range),each -:0last  NB. Two new iplans halving range & tolerance
range=.2&|.                    NB. Range of integration = 2-item vector
hr=.(1,0#)|,each(,each-:0+/)  NB. Makes two half-ranges from one
each=.,&.>                     NB. Each adverb = "under open"
```

and finally a verb which, for a given function *f*, realises the algorithm described above in words.

```

simf=.3 : 'if.(last y.)<|f simd range y.
          do. r=+>/simf each nip y.
          else.r=.f sim4 range y.
          end.'

f=.3 : '(y.)*1+1 o.y.'      NB. function f(x)=exp(x).(1 + sin x)

simf 0 1 0.000001          NB. integral of from 0 to 1 corr. to 5 d.p.
2.62761

```

As an aside, the steps taken by the adaptive algorithm can be traced by omitting the +/ from simf, and rerunning. simf is readily converted into a tacit recursive form:

```

simt=.stop`(+/@(>@($: each@nip)))@.test  NB. Recursion using gerund
stop=.f sim4@range                      NB. Stopping verb
test=.last<|@(<f simd@range)            NB. Stopping test

```

leading finally, by the technique described earlier, to an adverb in which f is parameterised.

```

adapt=.1 : 'x.sim4@range`(+/@(>@($: each@nip)))@.(last<|@(<f(x.simd@range)))'

":1.9 f adapt 0 1 0.000001      NB. Make J display 9 sig figs.
2.62761245

```

In summary, here are the full definitions of simp and adapt:

```

simp=.1 : 'swt +/ .*x.itogly'  NB. Simpson adverb, e.g. fn simp 0 1 4
swt=.%.30(subint * swts)      NB. Adjusts swts for 1/3.h
subint=.%.0(%'-/0|.)         NB. sub-interval for iplan=range, # of ints
swts=.3 : '1.((1+1(y.)$4 2),1' NB. Simpson weights for iplan y.
itogly=.0(grid@{.,'subint'(>:@last)/.)) NB. by turning iplan into grid
grid=.+'(+i.)//              NB. left-value, grid-unit, no of points
last=.{.,0|                  NB. last item in vector as scalar
adapt=.1 : 'x.sim4@range`(+/@(>@($: each@nip)))@.(last<|@(<f(x.simd@range)))'
simd=.1 : 'x.sim2-x.sim4'     NB. Difference between 2 and 4 subints.
sim2=.1 : 'x.simp%2'          NB. Simpson rule with 2 sub-intervals
sim4=.1 : 'x.simp%4'          NB. Simpson rule with 4 sub-intervals
range=.26{.                   NB. Range of integration = 2-item vector
nip=.{hr@range},each -:@last  NB. Two new iplans halving range & tolerance
hr=.{1.8%}|.each{.,each-:@+/) NB. Makes two half-ranges from one
each=.&.>                     NB. Each adverb = "under open"

```

References

- [1] *APL Operators and J*, Norman Thomson, Vector Vol.13, No. 1 pp.143-148
- [2] *Technical Note on Confidence Limits*, Norman Thomson, Vector Vol.12 No.4, pp.103-104
- [3] *J Phrase Book*, Chris Burke, distributed with J rel. 3, 1996

APL and J (3): Operators – Reduce, Scan, Outer Product

by Chris Burke

This series of articles entitled *APL and J* will try to explain the J way of thinking, by contrasting APL and J coding examples. The term APL will be used generically to refer to facilities found in most commercial implementations of APL.

For the next few articles, I will discuss operators. To start off with, we will take a look at J's use of operators as a whole, then look specifically at reduce, scan and outer product.

Housekeeping

As usual, we use the letters *V*, *M*, and *A* for vectors, matrices and arrays of any dimension, optionally followed by numbers to denote their shape. Thus *v* for any vector, *V5* for a vector of length 5, *M45* for a 4-by-5 matrix, *A245* for a 2-by-4-by-5 array. Where values are required, *V5* will be defined as `1 5`, *M45* as `4 5 p 20`, for example:

```

      M45
0   1   2   3   4
5   6   7   8   9
10  11  12  13  14
15  16  17  18  19

```

APL terminology will be used, except where J terminology is required for clarity. Since J uses origin 0 only, here `0io+0` is assumed throughout.

Overview

J makes much greater use of operators than APL, and this is probably the hardest aspect of J to grasp. For example, if you tried to write programs without using operators at all, in APL this would be a minor inconvenience, whereas in J you would be completely stuck – since all J programs are created with operators!

In general, while operators are common in APL, they typically occur in a relatively few expressions such as `+/` and `^.=` which become familiar and are used as if they were primitive functions. Thus the APL programmer may try to solve a problem by thinking solely in terms of the “functions” that can be brought

to bear on it. This approach will not work in J; or at least, if you try to think solely in terms of functions, you will find it tough going! Operators are essential in solving problems, and it is important in J that you develop a good understanding of how they work.

Moreover you not only use operators, you also need to be able to define them, whereas in APL it would be unusual to define new operators to solve problems. Indeed one major implementation, APL+Win, does not even allow you to define new operators.

This focus on operators in J can be a bit intimidating to the newcomer, who may think that programming with operators is difficult. True there is some learning involved, but once mastered, it becomes pretty simple. Moreover, it gives you a much clearer understanding of how APL works – a good reason to learn J is that it will teach you so much about APL!

Tacit Definition

An operator expression like `+/` is called a tacit definition, in that it defines a new function without referring to the new function's argument. Compare with *explicit* definitions which do refer to the function's argument (e.g. any APL function defined with the function editor). Tacit definitions occur frequently in both APL and J, though the terminology is new to J and can be a source of difficulty. I have heard experienced APL programmers say they do not understand tacit definition – clearly they are unaware that they use it all the time in APL.

Tacit definition may be clarified by giving the new function a name, which in APL is only possible with Dyalog APL (but which should be possible in all APLs):

<code>sum← +/</code>	Dyalog APL
<code>sum=. +/</code>	J

Using Dyalog's composition operator, it becomes easy to generate the kind of tacit definitions so often seen in J:

```

root← * ° 0.5
square← * ° 2
dist← root ° sum ° square
dist 5 12
13

```

When should you use tacit, and when explicit definition? Whenever you feel like it! For example the following APL function uses both tacit and explicit definition:

```

      ▽ r←dist dat
[1]   r←(+/dat*2)*0.5
      ▽

```

Is tacit definition better than explicit definition? It certainly can be – try writing an APL function that simulates `+/` without using tacit definition. Then modify this function to handle the general case of `f/`.

Now let's look at some APL operators.

APL Reduce

At first sight, reduce in J (called *insert*) seems to be the same as in APL:

```

      +/ 2 3 5 7          APL
17
      +/ 2 3 5 7          J
17

```

However, with a matrix argument, APL applies `+/` along the last axis, and J along the first axis:

```

      M34
0 1 2 3
4 5 6 7
8 9 10 11

      +/ M34          APL
6 22 38

      +/ M34          J
12 15 18 21

```

So `+/` in J is closer to APL's `+/` or `+/[0]`, i.e. plus reduce along the first axis. To apply `+/` in J along the last axis as in APL, apply it *rank 1*, using the rank operator:

```

      +/"1 M34
6 22 38

```

Since reduce applies a function between each element of a vector, you might think that `,/` is an identity on a vector. This is true in J, but not in APL:

```

      v5 -: ,/ v5          J
1

```

```

0      V5 = ,/ V5      APL

```

Instead, `,/` in APL encloses its result:

```

1      (⊂V5) = ,/ V5      APL

```

What is happening is that *insert* in J inserts the function between the *items* of its argument, whereas *reduce* in APL inserts the function between the *elements* of the *vectors along the last axis*, and encloses the result for each vector, so that:

```
f/ M34
```

means:

```

0 1 2 3 f 4 5 6 7 f 8 9 10 11      J
(0 f 1 f 2 f 3) (4 f 5 f 6 f 7) (8 f 9 f 10 f 11)      APL

```

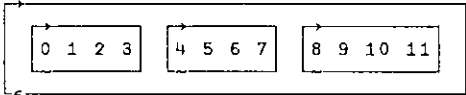
Strictly speaking, the latter is the APL2 definition of reduce, seen in APL2, APL+WIN and Dyalog APL. The treatment in SHARP APL is essentially that of J, except items are based on the last axis. It is interesting that APL does an implicit enclose. It is very often the case that an APL definition uses enclose, where J does not. For example:

```

,/ M34      J
0 1 2 3 4 5 6 7 8 9 10 11

display ,/ M34      APL

```



Since there is no difference in APL between a scalar and an enclosed scalar, the final result of an APL reduce may be simple. For example, `+/M34` is:

```
(0 + 1 + 2 + 3) (4 + 5 + 6 + 7) (8 + 9 + 10 + 11)
```

which is:

```
(6) (22) (38)
```

which is:

```
6 22 38
```

The effect of `, / M34` in APL can be produced in J using link insert:

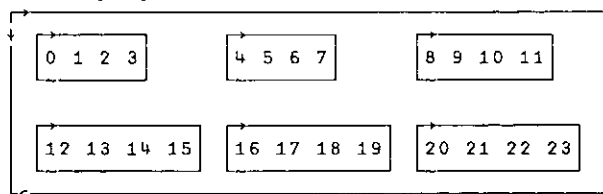
`; / M34`

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

The distinction between the way reduce applies to *vectors*, and the way insert applies to *items* becomes clearer with a 3D argument:

`display , / A234`

APL



`; / A234`

J

0	1	2	3	12	13	14	15
4	5	6	7	16	17	18	19
8	9	10	11	20	21	22	23

`, / A234`

J

```

0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
16 17 18 19
20 21 22 23

```

Replicate

In APL, the symbol `/` is used both for reduce and for replicate, whereas in J, replicate is given by a different symbol, `#` (copy).

APL Scan

Scan is called *prefix* in J, and is clearly not the same as in APL:

`+ \ 1 2 3 4`

J

```

1 0 0 0
1 2 0 0
1 2 3 0
1 2 3 4

```

The difference is that in APL, scan applies the verb with reduce, while J just applies the verb. Thus

$f \backslash 1\ 2\ 3\ 4$

means:

$(f / 1) (f / 1\ 2) (f / 1\ 2\ 3) (f / 1\ 2\ 3\ 4)$ APL

$(f\ 1) (f\ 1\ 2) (f\ 1\ 2\ 3) (f\ 1\ 2\ 3\ 4)$ J

Note that in the APL case f is dyadic, whereas in J, f is monadic.

In J, each application of f returns an item of the result, so that

$+\backslash 1\ 2\ 3\ 4$

is:

$(+1) (+1\ 2) (+1\ 2\ 3) (+1\ 2\ 3\ 4)$

J assembles these four items into a matrix, padding with zeroes as required.

To simulate APL scan, you need to use reduce explicitly:

$+\backslash 1\ 2\ 3\ 4$
1 3 6 10

The benefit of not having an automatic reduce in prefix, is that many operations do not require it. Such operations are straightforward in J, but may be awkward in APL. The following is a reverse scan in J – how would you do this in APL?

$<@!.\ 1.6$

0	1	0	2	1	0	3	2	1	0	4	3	2	1	0	5	4	3	2	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

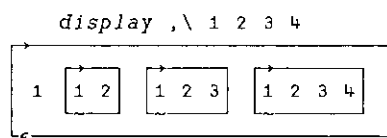
For another example, to enclose successive prefixes of a variable, you can use box prefix in J, but not enclose scan in APL.

$<\backslash 1\ 2\ 3\ 4$

J

1	1	2	1	2	3	1	2	3	4
---	---	---	---	---	---	---	---	---	---

Curiously, since $, /$ is an APL reduction that encloses its argument, it turns out that at least for a vector, $, \backslash$ performs an enclose scan!



Inverse

Some uses of prefix have inverses. For example, the inverse of $+\backslash$ is the first differences, and of $*\backslash$ is the successive ratios:

```
inverse=. ^: _1

+/\ inverse 1 3 6 10
1 2 3 4
*/\ inverse 2 3 5 7 11
2 1.5 1.66667 1.4 1.57143
```

Suffix

Suffix is analogous to prefix, except it applies to lists of elements taken from the end. Thus a reverse sum scan is:

```
+/\ 1 2 3 4      J
10 9 7 4

ϕ+ \ ϕ 1 2 3 4    APL
10 9 7 4
```

APL Outer Product

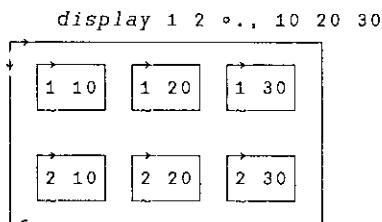
In J, the verb defined using insert may also be used dyadically (when it is called *table*), and is similar to APL's outer product.

```
1 2 3 +/ 10 20 30 40      J
11 21 31 41
12 22 32 42
13 23 33 43

1 2 3 °.+ 10 20 30 40     APL
11 21 31 41
12 22 32 42
13 23 33 43
```

Again, the definitions are not quite the same. Table in J applies the function between each cell of the left argument, and the entire right argument, whereas

outer product in APL applies the function between each pair of elements from the left and right argument. Consider the following APL example of catenating each pair of elements of two vectors:



You might expect that `, /` does the same thing in J, but it does not:

```
1 2 ,/ 10 20 30
1 2 10 20 30
```

This is because the rank of concatenate is unbounded, hence the left argument has only one cell (the entire argument), and this is applied to the entire right argument. The above expression is therefore the same as:

```
1 2 , 10 20 30
1 2 10 20 30
```

You can change this behaviour by using rank. For example, if both ranks are 0, then J will concatenate each pair of elements as in APL. The individual results are assembled in a simple array, rather than enclosed:

```
1 2 , "0/ 10 20 30
2 30
```

To duplicate APL's behaviour exactly, use box atop concatenate:

```
1 2 <@, "0/ 10 20 30
```

1 10	1 20	1 30
2 10	2 20	2 30

Alternatively, by setting the left rank to 0 and leaving the right rank unbounded, you can concatenate each element of the left argument to the entire right argument:

```
1 2 , "0 _/ 10 20 30
1 10 20 30
2 10 20 30
```

Leaving the left rank unbounded and setting the right rank to 0 catenates the entire left argument to each element of the right argument.

```

      1 2 , " _ 0/ 10 20 30
1 2 10
1 2 20
1 2 30

```

N-Wise Reduction

In APL2, a reduce may take a left argument that specifies the number of items to be used in each application of the function, called *n-wise reduction*, e.g.

```

      2 +/ 1 2 3 4 5 6
3 5 7 9 11

```

This is:

```
(1+2) (2+3) (3+4) (4+5) (5+6)
```

In J, this is handled by prefix (where it is called *infix*). As in APL2, the left argument specifies the number of items to be used. If positive, the items overlap; if negative, the items are non-overlapping. For example:

```

      2 +/\ 1 2 3 4 5 6
3 5 7 9 11
     -2 +/\ 1 2 3 4 5 6
3 7 11

```

As with prefix, the verb argument to infix need not be applied with reduce. For example:

```
-2 <\ 1 2 3 4 5 6
```

1	2	3	4	5	6
---	---	---	---	---	---

A neat way of creating a matrix with a given number of columns from a vector, is to use identity infix:

```

      -2 J\ 1 2 3 4 5 6
1 2
3 4
5 6

```

Reference

Iverson, K.E. *J Introduction and Dictionary*, Iverson Software, 1996

At Play With J - Extended Integers

by Eugene McDonnell (*eemcd@aol.com*)

Extended Integers

J has recently had added to it a new class of number, called *extended integer*. An extended integer can be produced by applying the extend verb *x:* to an ordinary integer. An extended integer is displayed as an ordinary integer terminated with an *x*, like this:

```
x: 1234
1234x
```

An extended integer may also be written directly by putting an *x* at the end of an ordinary integer:

```
1234x
1234x
```

If one or more of the integers in a list is extended, they are all extended:

```
1 2x 3
1x 2x 3x
```

Various primitives produce extended integer results if the argument is extended. For example, very large exact factorials are possible:

```
! 30x
2652528598121910586363084800000000x
*/ x: >: 1. 30
2652528598121910586363084800000000x
```

Some verbs *f* signal domain errors on some extended arguments because the result is not integral; however, *<. @f* and *>. @f* will work on extended arguments. I think you'll get the idea from a few examples:

```
1234 % 5x
|domain error
| 1234 %5x
1234 <. @% 5x
246x
1234 % 2x
617x
```

[illegible]

What if you want to turn an extended integer into an ordinary integer? Those of you conversant with J will guess that the way to do this is to apply the inverse of `x:`, like this:

```

      x: ^: _1 [ 1234x
1234

```

The Application

I was excited when I heard about extended integers, because I had an immediate use for them. Jeffrey Shallit gave a paper at APL83 in which he discussed the problem of determining how many times the random number generator had been used, given a value of the random link. In order to give an APL solution he had to include in the paper a portion of an extended arithmetic package. This was because the numbers needed to solve the problem were very large integers. Stating the problem as simply as possible, given the equation:

$$y = M|q \wedge x \quad \text{NB. (A)}$$

the problem is to find x , where y , M , and g are known. This is the basis of the random number generator we shall be discussing. The value g is also known as the generator, and this can lead to confusion when we talk about the generator of the generator. I offer my apology in advance. The problem is sometimes known as the logarithm problem, since (forgetting the M -residue for a moment), if we have:

$$y = g \wedge x$$

and know y and q , we can find x by taking the base- q logarithm of y :

$$x = q \wedge y$$

In solving (A) for the particular problem of APL\360 and its descendants (including J), x can be as large as 2,147,483,646. For this value of x , g^x has over 9,000,000,000 decimal digits, and would take several hundred large volumes to

print out. There are tactics one can employ to cut the size of the problem down, but extended-precision arithmetic will still be required. In J's implementation, the phrase `M&|@^` is recognized and can be computed efficiently both in time and space.

The technique discussed by Shallit to solve the problem is due to Pohlig and Hellman. You'll have to look up the references if you are interested in the mathematical background, since I shall focus on the problem's algorithmic aspects.

The Gory Details

Several global constants are needed. The modulus used in random number generators of the APL\360 kind must be a prime. The largest prime that can be stored as a 4-byte integer is in fact also the largest integer that can be stored, that is, one less than 2^{31} . This prime was discovered by Euler and for over a hundred years was the largest prime known. It is the Mersenne prime M_{31} , too, for those of you interested in the Euclidian perfect numbers.

```
M =. x: <: 2^31
```

It is convenient to have the value of the integer one less than M handy:

```
L =. <: M
```

For the random number generator to have maximum period, the generator g must be a primitive root of the modulus. A primitive root of a prime has the property that its powers, mod the prime, are distinct. For example, the prime 7 has 3 and 5 as primitive roots, because their powers, mod 7, are distinct:

```
7|3 5^/>:1.6
3 2 6 4 5 1
5 4 6 2 3 1
```

but the other positive integers less than 7 have repeated elements:

```
7|1 2 4 6^/>:1.6
1 1 1 1 1 1
2 4 1 2 4 1
4 2 1 4 2 1
6 1 6 1 6 1
```

Dr. Bryant Tuckerman, of the IBM Watson Research Laboratory in Yorktown Heights, New York, gave the APL\360 implementors the primitive root $7^{\wedge}5$, or

16807. A decade or so later people began exploring random number generators extensively, and were surprised to find that there was no better generator than this for the modulus M:

```
g =, x: 7^5
```

A prime-power factor is an integer all of whose factors are the same. For example, 32, 9, 125, 49, and 11 are all prime-power factors. The prime-power factors of L play a key role in the algorithm. The primitive q: in J yields the prime factors of a number, but these may be repeated. For example, q: 12 is 2 2 3. The algorithm requires that repeated primes be replaced by their product. The verb h, to be defined later, factors numbers and replaces repeated items by their product:

```
f =. h L NB. f is 2 9 7 11 31 151 331
```

Certain powers of the generator g are needed. Those needed are the quotients of dividing L by f, and multiplying this quotient by the integers less than the items of f. For example, for the factor 7 we get:

```
..B =. (L%7)*i.7
      0x
306783378x
613566756x
920350134x
1227133512x
1533916890x
1840700268x
```

and similarly for the other factors.

The verb p, to be defined later, raises the generator g to to any integer power, mod M. We use it to raise the generator to the powers B:

```
..C =. p (L%7) * i.7
      1x
1600955193x
894255406x
1205362885x
1752599774x
1537170743x
1599590586x
```

Such a list is made for each prime-power factor. These are boxed and joined together, forming `q`, a list of lists, containing 542 numbers altogether (+/2 9 7 11 31 151 331), and too large to display here.

```
q =. <@p@j"0 f
```

You should be warned that the formation of `q` takes a minute or so to execute, depending on the speed of your computer. I find it convenient to comment out this line in the script, and insert the value of `q` directly.

We need the quotient of `L` with its factors as a separate global noun:

```
.. e =. L % f
1073741823x
238609294x
306783378x
195225786x
69273666x
14221746x
6487866x
```

Those are all of the global nouns. Now we have to deploy a number of utility verbs. The verb `w`:

```
w =. -. ^ #/.
```

raises each item of its argument's nub to its tally:

```
w 2 2 2 3 3 5 7
8 9 5 7
```

The verb `h`:

```
h =. w @ q:
```

factors its argument and produces the prime power factors from it:

```
h L
2 9 7 11 31 151 331
```

The verb `s`:

```
s =. M&|^
```

raises its left argument `x` to the power of its right argument, mod `M`:

```

3 s 2
9x
16807 s 2000
75099568x

```

The verb p:

```
p =. g&s
```

raises g to the power of its argument:

```

p 2000
75099568x

```

The verb j:

```
j =. L&% * i.
```

divides its argument into L, and multiplies this quotient by the non-negative integers less than it.

```

.. j 7
0x
306783378x
613566756x
920350134x
1227133512x
1533916890x
1840700268x

```

The Main Problem

With all this behind us, we're ready to discuss the main problem. Suppose we find that the value of y, the random link, is 1209311799:

```
y =. 1209311799
```

We define a verb t:

```
t =. f"_ .. q"_ i.&> ] s e"_
```

The phrase

```
] s e"_
```

can be replaced by

```

      ,.D =. y s e NB. let this be (D)
2147483646x
473297587x
1537170743x
353622995x
      4096x
709324280x
667991092x

```

The heart of the matter is that each distinct value that y may take yields a different list D . We look for the index of each item of D in q , finding:

```

] E=. q i.> D
1 1 5 4 23 142 268

```

These values are the residues we seek. We form a table F by stitching f and E :

```

] F =. f ., E
2   1
9   1
7   5
11  4
31  23
151 142
331 268

```

and this is also the result of applying verb t to y :

```

t y
2   1
9   1
7   5
11  4
31  23
151 142
331 268

```

If you refer to Hui's article, you'll see that F is similar to the table shown on page 64, beneath the expression $c \text{ mr } q$. That is, F is a table of moduli and residues. For example, the items in list D correspond to the factors in f . In particular, the value $1537170743x$ corresponds to the factor 7. We gave all the possible values that may be taken on in this position in list C back a bit. In C we find that the value $1537170743x$ is in position 5, and so in F we find the value 5 next to the 7 in the first column. It is a remarkable fact that the expression $y \text{ s e}$ produces values which *must* occur also in the corresponding item of q , and that its index in the list in q is the residue we want for the next step.

The verb `r`:

```
r =. {: @ (cr1/ @ t)
```

inserts Hui's verb `cr1` between each of the items of its argument, and yields a two-item list, with the first item necessarily equal to `L`, and the second item the power of `g` yielding `y`, mod `M`.

```
x:z =. cr1/F
2147483646x 1234567x
L
2147483646x
```

We take the tail of `z` as our desired `x`:

```
] x =. {: z
1234567x
```

We can verify that `x` is indeed the desired value by applying `p` to it:

```
p x
1209311799x
y
1209311799x
```

Are we happy? We shouldn't be yet, because this wasn't precisely the problem we wanted to solve, which was, how many times had the random number generator been used to arrive at the given random link. This part is easy, because we know that the initial value of the random link is 16807, which corresponds to exponent 1. All we have to do to get the value we want is to decrease `x` by 1. This gives us at last the verb `ner`:

```
ner =. <:@r NB. number of executions of roll

x:ner y
1234566x
```

References

Hui, Roger K. W., The Ball Clock Problem, *Vector* 12 2, October 1995, 55-66

Pohlig, Stephen C., and Martin E. Hellman, An Improved Algorithm for Computing Logarithms Over $GF(p)$ and Its Cryptographic Significance, *IEEE Trans. Info. Theory* IT-24 (1978) 106-110

Shallit, J. O., Merrily We Roll Along: Some Aspects of ?, *APL83 Conference Proceedings, APL Quote-Quad* 13 3, March 1983, 243-248

Appendix

NB. Script for finding x in $y = m|g^x$, knowing y , M , and g ,
 NB. particularized for use in analyzing the behavior of
 NB. the linear congruential random number generator found
 NB. in APL\360 and its descendants.

NB. GLOBAL NOUNS

```
M =. x: <: 2^31
L =. <: M
g =. x: 7^5
f =. h L NB. f is 2 9 7 11 31 151 331
q =. <@p@j"0 f NB. comment this line out
NB. replace line above by value of q directly
e =. L % f
```

NB. UTILITY VERBS

```
w =. ~. ^ #/.
h =. w@q:
s =. M&|@^
p =. g&s
j =. L&% * i.
```

NB. HUI's CHINESE REMAINDER VERBS FROM VECTOR 12 2, P 66
 NB. INCLUDING GCD AS A LINEAR COMBINATION

NB. Chinese Remainder

```
ab =. |.@(gcd/ * [ % +./)@(&{.)
cr1 =. [: |/\ *.&{. , ,&{: +/ .* ab
chkc =. [: assert ,&{: -: ,&{. | {:@cr1
cr =. cr1 [ chkc
```

NB. GCD as a Linear Combination

```
g0 =. , ,. =@i.@2:
it =. {: ,: { - {: * <.@%&{./
gcd =. (}.@{.)@(&{it^:(*@{.@{.)^:_))@g0
```

```
assert=. 13!:8@(&{12"_)^:-.
```

NB. MAIN VERBS

```
t =. f"_ . q"_ i.&> ] s e"_
r =. {: @ (cr1/ @ t)
ner =. <:@r NB. number of executions of roll
```

Using a Defined Adverb to Trace Execution in Tacit Programs in J

by David Alis

Introduction

This article presents a J adverb that's useful in tracing through tacit programs. Unlike explicitly defined programs, there is no obvious way of seeing what happens when tacit programs execute. Small programs with only four or so primitives (e.g. `mean=. +/ % #`) are not too difficult to analyse. However it's quite easy to create monsters with upwards of 20 components. While you are composing the code things are clear and it's easy to see how the code works. Later on it's more difficult. Consider, for example, the set of scripts which accompany the PHRASES Help file. An analysis of the 52 files gives the result that of the 1093 phrases more than 20% have ten or more syntactic components. Top of the list are these two with 34 components each.

```
,: @i. `([[: ,/ 0a, . @($: &. <: ){"2 1 \:"1@=@i.)@.(1&<)
/: -@, @> @(* / &. > / ) @ (< @ (1&, ) @ (* / \) / . ~ ) @ q:
```

What do they do? How can we analyse and understand them?

Tracing Program Execution

In explicit programs you can trace execution using a function such as `SHOW=. (1!:2)&2`. Put it in strategic points in the program and display intermediate results. This version of `SHOW` displays its right argument on the screen which is then returned unchanged (a property of `1!:2&2`).

```
120+ SHOW i.3 4
0 1 2 3
4 5 6 7
8 9 10 11
120 121 122 123
124 125 126 127
128 129 130 131
```

If we rewrite the tacit program `mean` to display the results of each of its components we have:

```
mean=. (SHOW@(+/)) (SHOW@%) (SHOW@#)
```

```

      mean 2 3 45
3
50
16.6667

```

i.e. The 3 is the number of items, the 50 is their total, and the 16.6667 is the result.

A Very Simple Example

It is a surprise to see that the even simpler example of `Add` takes you straight to the heart of the semantics of the J language.

```

Add=.SHOW@+
  2 2 3 4 Add 9
11
11
12
13
11 11 12 13

```

Why has `SHOW` been called four times? The reason is that dyadic `+` has rank zero which means each element in the argument will be treated individually. Using `Add` monadically we see that `Add` is called only once because monadic `+` has infinite rank.

```

Add 2 2 3 4
2 2 3 4
2 2 3 4

```

We can find the rank of functions by using the `b.` adverb,

```

+ b. 0
- 0 0
  SHOW b. 0
- - -
  Add b. 0
- 0 0

```

A simple program like `SHOW` is not quite sufficient to debug and trace an explicit function. Appendix 1 shows a slightly extended form used to trace the working of the `rplc` program in the `STRINGS.JS` file.

Using an Explicitly Defined Adverb

`SHOW` is clumsy to use with tacit programs because it's a verb. Converting it to an adverb with the `@` conjunction helps, but only a bit: `SHOW=. (1!:2)&2 @`

Using it we would redefine mean as follows:

```
mean=. (+/SHOW)(%SHOW)(#SHOW)
```

The main drawback with this version of `SHOW` is that it only shows function results and not their arguments. The way round this is use an explicitly defined adverb. The following version of `SHOW` displays the arguments to the verb instead of the results (or, using the commented expression, both results & arguments).

```
SHOW=. 1 : 0
show=. 1!:2&2
(x.@show) : ((show@[ x.( show@)])
NB.(show@x.@show) : ( show@((show@[ x.( show@)])) NB. results + args
)
mean=. (+/SHOW)(%SHOW)(#SHOW)
mean 2 34 45
2 34 45      NB. the argument to #
2 34 45      NB. the argument to +/
3            NB. the right hand argument to %
81           NB. the left hand argument to %
27
```

Coercing the Explicit Adverb to Keep Rank

However, caution is needed because the adverb has a significant but easily overlooked influence on the rank of the verbs it operates on. If we redefine `Add=. +SHOW` then although dyadic `Add` gives the correct result something has changed.

```
3 4 5 Add 8 9 10
8 9 10
3 4 5
11 13 15
```

It is called only once instead of three times as before. To make it work properly the rank conjunction is used to coerce `Add` to have the same rank as `+`.

```
Add=. (+SHOW)" +
3 4 5 Add 800 900 1000
800
3
900
4
1000
5
803 904 1005
```

The `" +` has forced `Add` to be called as many times as elements in the argument.

Agreement and Rank

This simple `Add` function also shows how agreement works when dyadic functions are called with different-shaped arguments. For example we know that `100 200 + i.2 4` produces

```
100 101 102
203 204 205
```

because the rules for agreement are satisfied. When there is agreement then `J` re-structures the smaller array to have the same size as the larger.

"In the phrase `p v q`, the arguments of `v` must agree in the sense that their frames (relative to the ranks of `v`) must be a prefix of the other. [J-Dict]"

The `SHOW` adverb illustrates this better using a non-zero rank verb. In the following example `Add` is redefined to be rank-1 and the inter-relationship between agreement and rank can be seen in the example.

```
Add=. +SHOW"1
```

When two arrays each of shape `2×3×4` are `Add`'ed together the program is called 6 times.

```
x=.(2 3 4$i.24) Add 1000*>:i.2 3 4)
1000 2000 3000 4000
0 1 2 3
5000 6000 7000 8000
4 5 6 7
9000 10000 11000 12000
8 9 10 11
13000 14000 15000 16000
12 13 14 15
17000 18000 19000 20000
16 17 18 19
21000 22000 23000 24000
20 21 22 23
```

However, when we try to add a `2×3` matrix to a `2×3×4` array execution starts but cannot complete.

```
x=.(2 3$i.6) Add (1000*>:i.2 3 4)
1000 2000 3000 4000
0 1 2
|length error
| x=.(2 3$i.6)      Add (1000*>:i.2 3 4)
```

Execution started because the rank of Add is 1 so that J determined that there was agreement. The 2 columns of the left hand matrix are in agreement with leading 2 dimensions (i.e. 2×3) of the right hand array. However, once it began executing it immediately signalled a length error when it tried to add a four-element vector to a three-element vector.

Defining Add to be a string concatenation would show more clearly how agreement works because the length error is avoided; e.g. Add=. ; SHOW"1

```
x=((2 9$'abcdefghijklmnopqr') Add 2 3 4$'ABCDEFGHJKLMNOPQRSTUVWXYZ')
```

Generalised Form of the SHOW Adverb

This final version of SHOW is more general. It decorates the arguments and also adds a representation of the calling verb. It displays the arguments and the results; it also handles the inverse of the verb so that it can be used with the dual & conjunction.

```
SHOW=. 1 : 0
NB. utilities
show =. 1!:2&2
showm =. > 0 {; 0 show 0 ('          monadic-'&);
showdl =. > 0 {; 0 show 0 ('          dyadic left-'&);
showdr =. > 0 {; 0 show 0 ('          dyadic right-'&);
showmi =. > 0 {; 0 show 0 ('          inverse monadic-'&);
showdli =. > 0 {; 0 show 0 (' inverse dyadic left-'&);
showdri =. > 0 {; 0 show 0 (' inverse dyadic right-'&);
NB. operate on verb
G=.5!:1 <'x.'      NB. gerund representation of verb
AA=.G&,0<         NB. the verb AA decorates the output
monadic =. show@x.@showm @ ( AA f.)
dyadic =. show@((showdl@ (AA f.)@[] x. (showdr@ (AA f.)@[]))
verb=.monadic : dyadic f.
NB. operate on the inverse
INV=.x.^:_1        NB. The inverse verb
G=.5!:1 <'INV'
BB=.G&,0<
monadic=.INV@showmi @ ( BB f.)
dyadic=.show@ ( ( showdli@ (BB f.)@[] INV ( showdri@ (BB f.)@[] ) )
inverse=. monadic : dyadic f.
z=((verb"x.) :. (inverse"INV)) f.
)
```

Example:

```
fac=. (1: `{ }* ($: SHOW)@<:))@. (1:<])
fac 7
monadic- $: 6
monadic- $: 5
monadic- $: 4
monadic- $: 3
monadic- $: 2
monadic- $: 1
1
2
6
24
120
720
5040
```

Appendix:

This is an example of how to use a simple SHOW function to trace execution in an explicitly defined program. It is taken from the distributed STRINGS.JS script file. The rplc program has been altered to trace execution:

```
rplc=: 4 : 0
NB. replace characters in text string
NB. syntax: text rplc old;new
SHOW=. 3 : (> '1!2&2 y.' ; ' ' ; ' 1!2&2 x.;y.' ; 'y.')
'on'=. ,&.>y.
l=. #o
c=. o E. x=. ,x.
if. (0<l) *: 1 e. c do. ('string not found or empty'SHOW x) return. end.
bx=. #1.@#
('c= ' &SHOW )c=. bx c
while. ('Result of while test=' &SHOW ) 0 e. d=. 1,<:\(#o)<:({.-):) c
do. ('c becomes= ' &SHOW ) c=. ('Value of d=' &SHOW ) d#c end.
('p=. (i.#x) e. 0,c NB. p= ' &SHOW )p=. (i.#x) e. 0,c
('x=. p <:;1 x NB.x= ' &SHOW )x=. p <:;1 x
('b=. o&-@ (l&$) &> x NB. b= ' &SHOW )b=. o&-@ (l&$) &> x
f=. n&,@ (l&.) &.>
SHOW 'evaluation of components in the expression',(10{a.),'(f b#x) (bx b) }x'
;((('f b#x) ' &SHOW ) f b#x) ((' (bx b) ' &SHOW ) bx b) }x
)
```

Here is the result:

'ababababab' rplc 'aba'; '=ABCDEF='

c=	0 2 4 6
----	---------

Result of while test=	1
-----------------------	---

Value of d=	0 4 6
-------------	-------

c becomes=	0 4 6
------------	-------

Result of while test=	1
-----------------------	---

Value of d=	0 4
-------------	-----

c becomes=	0 4
------------	-----

Result of while test=	0
-----------------------	---

p=. (i.#x) e. 0,c NB. p=	1 0 0 0 1 0 0 0 0 0
--------------------------	---------------------

x=. p <;.1 x NB. x=	abab ababab
---------------------	-------------

b=. o&-:@(l&\$) &> x NB. b=	1 1
-----------------------------	-----

evaluation of components in the expression
;(f b#x) (bx b) }x

(bx b)	0 1
--------	-----

(f b#x)	=ABCDEF=b	=ABCDEF=bab
---------	-----------	-------------

=ABCDEF=b=ABCDEF=bab

Index to Advertisers

Causeway Graphical Systems Ltd	4
Dyadic Systems Ltd	17
APL Booklist (Renaissance Data Systems)	36
Strand	6
Uniware	2
Vector Back Numbers	5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Compuserve: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Duncan Pearson,
Causeway Graphical Systems Ltd
The Maltings, Castlegate
MALTON, North Yorks, YO17 0DP
Tel: +44 (0) 1653-696760
Email: 100265.1564@compuserve.com

Authors wishing to use Word for Windows may contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Tel: +44 (0) 1439-788385
Email: 100331.644@compuserve.com

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1996/97 Committee

Chairman	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary	Sylvia Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	Duncan Pearson 01653-696760 100265.1564@compuserve.com	Causeway Graphical Systems Ltd, The Maltings, Castlegate, MALTON, North Yorks YO17 0DP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 100257.1756@compuserve.com	138 Burton Stone Lane, YORK YO3 6DF
Education	Dr Ian Clark 01388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Projects and Publicity	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Duncan Pearson	Causeway Graphical Systems Ltd (01653-696760)
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (01302-539761), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths (0171-289 0471), David Ziemann (0181-348 4039), Jon Sandles (01904-612882), Ajay Askoolum	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Mallings, Castlegate,
MALTON, North Yorks YO17 0DP
Tel: 01853-696760
Fax: 01853-697719
Email: causeway@compuserve.com
Web: www.causeway.co.uk

Compass R&D Ltd
10 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 01483-302248
Fax: 01483-302279

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0171-353 8900
Fax: 0171-353 3325
Email: 100020.2632@compuserve.com

Insight Systems Aps
Nordre Strandvej 119A
DK-3150 Hellebæk
Denmark
Tel: +45 49 78 20 20
Fax: +45 49 76 20 30
Email: info@insight.dk
Web: www.insight.dk

APL2000
6610 Rockledge Drive
Suite 502
Bethesda MD 20817 USA
Tel: +1 (301) 564-5020
Fax: +1 (301) 564-5021
Email: sales@apl2000.com
Web: members.apl.com/APL2000

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 0171-922 8886
Fax: 0171-928 1006
Email: microapl@microapl.demon.co.uk
Web: www.microapl.co.uk

Soliton Associates Ltd
Groet Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337